

分类号

学校代码 10487

学号 M202073146

密级

华中科技大学

硕士学位论文

(学术型☒ 专业型☐)

面向深度神经网络的精细粒度动态 内存管理方法

学位申请人：龙子吟

学科专业：计算机系统结构

指导教师：华宇教授

答辩日期：2023年5月21日

**A Dissertation Submitted in Partial Fulfillment of the Requirements
for the Master Degree in Engineering**

**Fine-grained Dynamic Memory Management for Deep
Neural Network**

Candidate : LONG Ziyin
Major : Computer Architecture
Supervisor : Prof. HUA Yu

Huazhong University of Science and Technology
Wuhan 430074, P. R. China
May 21, 2023

摘要

图形处理单元(Graphics Processing Unit, GPU)具有强大的计算能力而能够支持深度神经网络模型的训练。为了达到更高的训练准确度,深度神经网络模型的深度和广度越来越高,而有限的 GPU 内存容量限制了深度神经网络的进一步发展。通常,内存交换和重计算技术被用于解决 GPU 内存不足的问题,这两种技术在前向传播过程中转移或丢弃了部分中间结果,被释放的内存空间则可以用于更深层次的计算。为了减少两种技术进行内存优化时产生的时间开销,现有的工作基于不同类型的特征提出了基于层的粗粒度决策方法。相关方法采用固定的决策方案增加了不必要的时间开销,同时内存交换方法因额外占用了 GPU 的内存空间而影响了决策的正确性。

针对以上问题,提出了面向深度神经网络的精细粒度动态 GPU 内存管理方法 FDNN (Fine-grained Dynamic Memory Management for Deep Neural Network)。FDNN 根据内存交换和重计算方法的性能开销分析,给出了不同批处理大小情况下训练速度最快的内存交换和重计算决策方案。在此基础上,根据内存交换方法移动数据的特点,FDNN 在内存交换方法的额外内存占用成为优化瓶颈时更新决策方案,保证决策结果的正确性。此外,对于内存交换方法带来的内存碎片问题,FDNN 将长期保留在 GPU 内存中的数据与即将换出的数据分开放置,减少了内存碎片发生的概率,进一步减少了内存空间占用。

实验结果表明,FDNN 可以在有限的 GPU 内存中支持训练更广更深的模型,同时保证了高训练速度。与现有的 GPU 内存管理方法 vDNN, IPABN 和 SuperNeurons 相比,FDNN 能训练的批处理大小平均提升了 2.13、1.73 和 1.59 倍,训练速度最高能提升 264%、38%和 56%。

关键词: 内存管理; 重计算; 内存交换; 深度神经网络

Abstract

Graphics Processing Units (GPUs) are capable of supporting the training of deep neural network (DNN) models due to their powerful computing capabilities. To achieve higher training accuracy, deep neural network models are getting deeper and wider, and the limited GPU memory capacity limits the further development of deep neural networks. Usually, memory swapping and recomputation techniques are used to solve the GPU memory shortage problem. These two techniques shift or discard some intermediate results during forward propagation, and the freed memory space can be used for deeper computation. To reduce the time overhead, existing work proposes layer-based coarse-grained decision methods based on the characteristics of different types of layers. The related methods add unnecessary time overhead by using a fixed decision scheme, while the memory swapping methods affect the correctness of the decision by taking extra GPU memory space.

The FDNN (Fine-grained Dynamic memory management for Deep Neural Network) is proposed to address these problems. Based on the performance overhead analysis of memory swapping and recomputation methods, FDNN gives the fastest memory swapping and recomputation decision scheme for different batch sizes. In addition, FDNN updates the decision scheme when the extra memory footprint of the memory swapping method becomes an optimization bottleneck according to the characteristics of the memory swapping method to move data, and ensures the correctness of the decision results. Moreover, for the memory fragmentation problem brought by the memory swapping method, FDNN places the data retained in GPU memory for a long time separately from the data to be swapped out, reducing the probability of memory fragmentation and further reducing memory space footprint.

Experimental results show that FDNN can support training wider and deeper models with limited GPU memory, while ensuring high training speed. Compared with the existing GPU memory management methods vDNN, IPABN and SuperNeurons, FDNN can train batches of 2.13, 1.73 and 1.59 times larger in size on average, and can train up to 264%, 38% and 56% faster.

Key words: Memory Management; Recomputation; Memory Swap; Deep Neural Network

目 录

1 绪论.....	1
1.1 研究背景与意义.....	1
1.2 国内外研究现状.....	5
1.3 研究目标和主要工作.....	9
1.4 全文结构.....	10
2 研究基础与相关技术	11
2.1 深度神经网络训练模型.....	11
2.2 支持 DNN 的内存交换技术.....	12
2.3 支持 DNN 的重计算技术.....	14
2.4 DNN 训练中的内存碎片问题研究.....	15
2.5 本章小结.....	16
3 精细粒度的动态内存管理方法设计与实现	17
3.1 总体设计.....	17
3.2 精细粒度张量选择方法.....	19
3.3 动态张量选择方法.....	32
3.4 张量放置策略.....	40
3.5 精细粒度的动态内存管理方法实现	42
3.6 本章小结.....	45
4 性能测试与结果分析	46
4.1 测试环境.....	46
4.2 对比方案与测试用例.....	46
4.3 张量选择结果及性能分析	48
4.4 本章小结.....	55
5 总结与展望.....	57
5.1 论文总结.....	57
5.2 展望.....	58
参考文献.....	60

1 绪论

本章首先阐述了面向深度神经网络的内存管理方法研究的背景及意义，其次介绍了国内外关于神经网络内存管理方法的相关研究概况，并提出了本文的研究目标和主要工作，最后介绍了全文组织结构。

1.1 研究背景与意义

深度神经网络(Deep Neural Networks, DNN)^[1]在工业界有广泛的应用，例如图像分类^{[2][3]}、物体检测^{[4][5]}、自然语言处理^{[6][7]}和语音识别^{[8][9]}等，这些深度学习应用在高性能计算和大容量内存方面都有需求^[10]。现在有几种新兴的异构硬件提供了良好的计算能力，如图形处理单元(Graphics Processing Unit, GPU)、专用集成电路(Application Specific Integrated Circuit, ASIC)和张量处理单元(Tensor Processing Unit, TPU)^[11]等，这些硬件加快了 DNN 模型的训练速度。为了达到更高的学习精度，DNN 模型的深度和广度越来越高，例如 VGG-16^[12]，ResNet-50^[13]，Inception v4^[14]，DenseNet^[15]和 GPT-3^[16]等，这些深层的 DNN 训练模型对 GPU 等硬件提供的有限内存空间提出了挑战^[17]。例如，GPT-3 模型包含 1700 亿参数，内存使用量达到了 TB 级别。然而，当前 NVIDIA GPU A100 的内存最大只能达到 80 GB^[18]，并且用于商业云中的主流 GPU 往往只有 16 GB 的内存，这些有限的内存限制了 DNN 架构的进一步探索和发展。

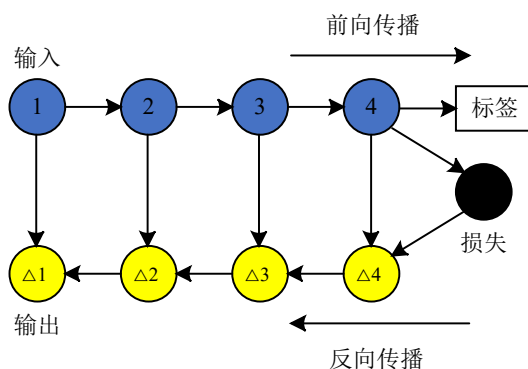


图 1-1 一次迭代过程的数据流图示例

在深度神经网络训练的过程中,内存空间占用主要包括^[19]:特征图(即前向传播的中间结果)、梯度图(即反向传播的中间结果)和卷积工作区(即卷积算法需要的额外内存空间)。除此之外,模型的权重也占用了部分内存空间,但相较于前三种,消耗的内存空间可以忽略不计。在三种主要内存空间占用中,梯度图和卷积工作区属于临时内存空间占用,这部分内存空间占用会在计算完成后立即释放,而特征图会在前向传播过程中被缓存,并在反向传播过程中用于计算每一层的梯度和权重的梯度。训练中一次迭代过程的数据流图如图 1-1 所示,其中蓝色节点为特征图,黄色节点为梯度图,箭头指向表示数据依赖关系。特征图除了在前向传播中用于下一个特征图的计算之外,还需要在反向传播中计算梯度图时复用,需要保存在 GPU 内存中。图 1-2 展示了特征图在内存中的占用时间。DNN 训练过程中的数据访问呈现对称的数据重用模式^[20],具体来说,前向传播与反向传播对特征图的访问按相反的顺序执行,前向传播中最先被访问的特征图在反向传播时最晚被访问,越早生成的特征图在内存中占用的时间越长。

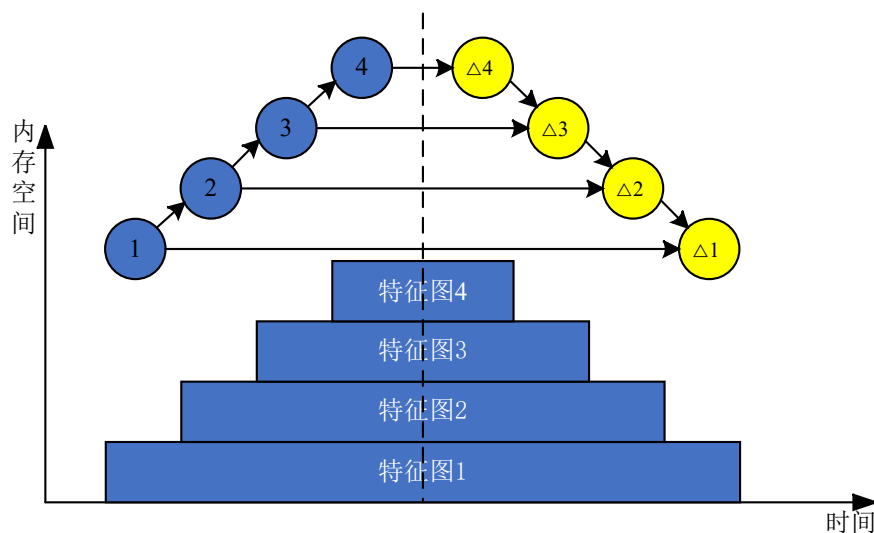


图 1-2 特征图内存占用时间

针对 GPU 内存不足的问题,现有工作通常使用内存交换和重计算方法^{[21][22]},利用 DNN 训练规律的数据重用模式,在前向传播过程中释放特征图的内存空间(为方便解释说明,本文将该操作称为逐出特征图操作),在反向传播中重新生成中间特征图,两者的区别在于如何进行重新生成。具体来说,内存交换将主存视为一个较大的

外部存储器，前向传播中将特征图换出至主存，反向传播再次访问该特征图时从主存换入该特征图。重计算则是在前向传播中将特征图丢弃，反向传播再次访问该特征图时基于 DNN 数据流图中特征图之间的数据依赖关系，将保存在 GPU 中的特征图作为计算源，重新计算出该特征图。

这两种方法虽然都能解决内存容量不足的问题，但会产生额外的时间开销。对于内存交换方法，由于 PCIe 总线的带宽有限，数据在 GPU 和 CPU 之间的传输速度较慢，而计算的速度较快，如果交换大量特征图到主存，当需要某特征图进行计算时，特征图可能还未拷贝至 GPU 内存，造成了计算的暂停等待。对于重计算方法，虽然 GPU 支持大量的并行计算，但在 GPU 内存不足时，计算资源也会随之消耗殆尽，重计算方法不可避免地产生了因重复计算带来的额外时间开销。在极端情况下，使用这两种方法带来的时间开销是正常迭代时间的数倍甚至数十倍，对于一次训练进行数百万次迭代的神经网络训练而言，增加的时间开销是巨大的。

为了减小两种方法带来的额外时间开销，现有的工作根据模型数据流图 and 不同类型层的特征提出了基于层的 GPU 内存管理方法。例如，vDNN^[23]选择计算时间长的卷积层的输入作为内存交换的对象，目的是增加内存交换与计算的时间重叠，减少计算暂停等待的时间。SuperNeurons^[24]选择卷积层输入进行内存交换的同时，选择计算时间较短且内存空间占用量大的激活层、池化层和批处理归一化层的输入进行重计算，目的是减少时间开销的同时更多地减少内存空间占用。然而，神经网络的一个层是由许多低级数学函数组成的高级计算抽象，每一层都包含成百上千个计算节点，即使是同种类型的层，也可能因其计算节点数量的不同表现出不同的特征。此外，不同的批处理大小(batch size, 即一次迭代输入的样本数量)和硬件环境也会使内存交换和重计算方法产生不同的内存和时间开销。因此，基于层的粗粒度内存管理方法无法基于不同的环境条件给出时间开销最小的选择方案，限制了 GPU 内存的优化空间。具体来说，基于层的 GPU 内存管理方法的问题分为以下三点：

(1) 不变的决策方案。粗粒度决策方案对于任意内存空间需求都提供同一个解决方案，造成不必要的时间开销。对于不同的模型和批处理大小，训练所需的内存空间大小不同，例如 VGG16 模型在批处理大小为 240 时所需的内存空间为 16.5 GB，对

于 16 GB 的 GPU 内存，只需选择大于 0.5 GB 的特征图进行内存交换或重计算即可解决内存不足的问题。假设选择 0.7 GB 的第三层输入特征图进行内存交换，一次迭代的训练时间为 2136.48 ms，但基于层的粗粒度内存管理方法(例如 SuperNeurons)只根据层的类型选择被逐出的特征图，选择多个特征图进行内存交换和重计算，将内存空间占用降到了 12.05 GB，一次迭代的训练时间为 2307.69 ms，增加了额外的时间开销。

(2) 无法量化的性能开销。粗粒度决策方案仅根据不同类型的层表现出来的特征进行决策，没有量化两种方法的内存开销和时间开销，无法确定特征图的选择优先级，也无法比较内存交换和重计算方法的性能开销。特征图的选择方面，不同特征图大小和进行内存交换的时间点都会影响内存交换方法的额外时间开销。假设某个卷积层处在前向传播的末尾，且其输入特征图较大，进行内存交换的时间较长，而根据 DNN 训练的数据重用模式，该特征图在反向传播时会很早被访问，那么此时需要等待特征图从主存复制到 GPU 内存中，产生了很长的计算暂停等待时间，因此并不是所有卷积层都适合进行内存交换。重计算方法的时间开销会随着决策的改变而改变，当某一特征图的父节点特征图被选择进行逐出时，该特征图的重计算时间会发生变化，因此不能仅根据某一层的计算时间对特征图做出选择。内存交换和重计算方法的选择方面，内存交换方法可以在计算的同时进行内存交换，内存交换与计算的时间可以重叠，从而隐藏部分内存交换方法的额外时间开销；重计算方法在反向传播时重新计算出的结果无需存放到 GPU 内存中，而是存放在寄存器中，直接用于梯度图的计算。粗粒度的选择方法没有充分利用两种方法的性能优势，进行性能开销最小的决策。

(3) 内存空间占用动态变化影响决策结果的正确性。根据 DNN 训练的数据重用模式，保存所有特征图时，内存空间占用的峰值出现在反向传播的起始阶段。内存交换方法换入特征图到 GPU 内存中需要分配额外的内存空间，假设该特征图较大，换入该特征图时，内存空间占用包括梯度图和换入的特征图，可能会造成新的内存空间占用峰值，因此，并不是对任意张量进行内存交换都可以减少 DNN 训练的最高内存空间占用。此外，由于特征图需要等到内存交换完成后才能释放其内存空间，这一延迟

释放的性质可能影响内存交换方法的正确性。

除了以上三点,由于内存交换方法打乱了特征图分配和释放的顺序,在前向传播过程中提前释放的内存空间可能由于较小而无法分配给其他特征图,造成了内存碎片问题。

针对以上问题,本工作提出了基于张量的精细粒度动态内存管理方法(张量即特征图和梯度图的数学表示),分析了内存交换和重计算方法的性能优势,比较了每个张量的内存交换和重计算方法的性能开销,在减少相同的内存空间占用的前提下,选择了时间开销最小的决策方案。此外,针对内存碎片问题,本工作还提出了张量放置策略。

1.2 国内外研究现状

深度神经网络应用的迅速发展增加了对大容量 GPU 内存的需求,但现有的 GPU 内存容量有限,这限制了对于更先进深度神经网络模型的探索。现有的工作根据 DNN 训练过程中规律的数据重用模式以及中间结果的数据特点,采取不同的内存调度和数据处理方法,使得深度神经网络训练能充分利用 GPU 等硬件设备的计算和内存资源,加速大规模深度神经网络在单个 GPU 设备中的训练速度。

目前国内外有很多工作致力于降低单个设备上神经网络训练的内存空间占用,基本上可以分为三个方面:(1)基于内存交换方法的研究。内存交换方法即将暂不使用的数据换出到其他存储设备中(例如主存、固态硬盘),当再次访问该数据时换入内存;(2)基于重计算方法的研究。重计算方法即在前向传播过程中将中间结果释放,而在反向传播过程中重新计算生成被释放的中间结果;(3)基于模型压缩方法的研究。压缩方法即在训练过程中利用压缩算法对模型参数或结构进行压缩。

1.2.1 基于内存交换的内存管理方法研究

内存交换方法通过换出部分数据降低内存空间占用。在换出数据的选择上,主要分为两种方法:一是训练前根据模型数据流图观察内存空间占用情况进行选择;二是训练中动态获取内存空间占用情况进行选择。

Chen 等人^[25]提出了一种基于通用数据流图的 GPU 内存优化策略。Tensorflow^[26]

的训练框架使用数据流图记录一个模型在每一次迭代中的训练任务，节点代表计算，边代表张量，即特征图。基于对数据流图的分析，选择生命周期较长的张量交换到主存中，降低数据传输带来的通信开销。此外，在反向传播过程中，过早地将特征图预取到 GPU 内存中会占用额外的 GPU 内存，而过晚地将特征图预取到 GPU 内存中则会增加训练时间开销，因此需要根据特征图的大小和计算时间选择合适的时间将特征图预取到 GPU 内存中。在完成预处理阶段后，优化的数据流图可以直接放在 Tensorflow 框架中进行训练，此方法适用于任何深度神经网络模型。Chen 等人^[27]使用启发式的调度算法实现了内存交换方法。由于不同的卷积算法需要的卷积工作空间不同，因此需要根据当前的内存空间占用情况选择合适的卷积算法，此方法为动态调度算法，即在训练时根据当前的资源使用情况选择不同的调度方法。Jin 等人^[28]提出了一种以层为中心的数据放置策略，跟踪记录每一层输出数据的访问顺序和被其他层数据引用的引用计数，当引用计数为 0 时换出数据，避免换出即将被访问的数据。Peng 等人^[29]提出了基于张量的 GPU 内存管理方法，该方法通过跟踪训练时的张量访问模式动态调整内存管理决策，在减少 GPU 内存空间占用的同时大幅减少训练时间。由于内存交换方法会产生多次内存分配和释放，一般来说，一个完整的训练包含数百万次迭代，如此频繁的内存操作会带来很大的性能开销。Wang 等人^[24]创建了一个内存池，提前分配一块 GPU 内存，减少内存操作的次数，此方法还提出了一个张量缓存，将访问频次高的张量放在张量缓存中，从而减少通信开销。Bae 等人^[30]提出了将数据换出到闪存中的方法 FlashNeuron，该方法使用 P2P 的方式实现了 GPU 内存与固态硬盘(采用 NVMe 接口)的直接通信，此外，该方法进行决策时首先换出所有张量，再根据张量大小和在网络中的位置决定其是否保存在内存中。

内存交换方法利用了空闲的存储资源，在减少对有限 GPU 内存的占用的同时，减少了整体训练时间，有效地提升了 GPU 内存利用率。此方法利用计算与通信的并行，将 CPU 和 GPU 之间的通信与计算时间重叠，但 PCIe 总线带宽有限，选择较多特征图数据换出仍会造成较大的时间开销。此外，由于换入特征图时仍需要为其分配内存空间，因此内存交换能够减少的内存空间占用有限。

1.2.2 基于重计算的内存管理方法研究

重计算技术利用深度神经网络的计算图通用且透明的特点，在前向传播的过程中释放特征图占用的内存，在反向传播需要使用该特征值进行计算时，根据深度神经网络计算图给出的依赖关系，通过重新计算的方式生成该特征值。

Gruslys 等人^[31]使用动态规划的方法来平衡缓存中间结果和重新计算的性能开销，对于用户指定的任何内存预算，都能找到最小化计算成本的最佳执行策略。在深度神经网络中的重计算方法通常有两种，即以速度为优和以内存为优的重计算方法。具体而言，以速度为优的重计算方法将重新计算出的张量保存在 GPU 内存中，计算其他张量时可以依据该张量进行计算。以内存为优的重计算方法不保存重新计算的张量，而是在每次需要该张量时都进行重新计算。Wang 等人^[24]提出了开销感知型重新计算方法，将这两种方法进行平衡，实现的最高内存空间占用与以内存为优的方法的内存空间占用相同，而额外的计算开销与以速度为优的方法产生的计算开销相当。Kusumoto 等人^[32]在固定内存预算情况下使用图论方法最小化重计算开销，并提供了相应动态规划解决方案。与之前的方法相比，该方法适用于大多数类型的神经网络模型。Peng 等人^[29]提出了基于张量的 GPU 内存管理方法。该方法根据训练中内存空间占用的变化动态调整张量访问模式。不同于在固定预算下减少计算开销的重计算方法问题研究，Jain 等人^[33]讨论了 DNN 训练时间和内存需求之间的权衡问题，提出了 Checkmate 方案，将权衡问题转化为张量重构的优化问题，并通过混合整数线性规划进行理论分析，得到高效的解决方案。

重计算方法利用计算资源置换内存资源，而深度神经网络中不同层的计算时间和计算结果大小不同，当计算时间越小，计算结果占用内存越大时，重计算方法的获得的收益越高，即以最少的计算资源置换最多的内存资源。当选择较多特征图进行重计算时，重计算的计算路径可能会变长，显著增加重计算的时间开销。

1.2.3 基于模型压缩的内存优化方法研究

DNN 训练中的模型压缩^{[34][35]}利用 DNN 参数和结构的冗余性，通过压缩算法对模型进行压缩，得到参数量更少、结构更精简的模型，减少训练所需的 GPU 内存空

间占用空间。模型压缩的方法主要分为压缩参数和压缩结构两类。压缩参数是指删除网络中的冗余参数、用较低位宽表示网络参数或利用矩阵聚类等方法映射网络参数。压缩结构则是指构造特殊的过滤器、网络层或者网络结构。

由于卷积层和全连接层的输入和输出之间存在大量冗余连接, Han 等人^[36]删除了神经元连接权值的范数值小于指定阈值的连接, 并通过轻量重新训练的方式恢复性能。为了避免因删除重要连接带来的高开销重新训练, Guo 等人^[37]提出了动态网络手术(Dynamic Network Surgery, DNS), 用于恢复被误删的重要连接。Wang 等人^[38]构建了面向高效 CNN 的通用过滤器, 通过主过滤器派生二级过滤器的方法减少了不同过滤器的内存空间占用, 并通过整合不同感受域的信息增强过滤器的能力。Prabhu 等人^[39]使用兼具稀疏性和良好连接性的图模拟卷积神经网络过滤器之间的连接, 并用于设计比现有 ResNet 和 DenseNet 性能权衡更好的 X-Net 网络。Jain 等人^[40]提出了一种特殊编码机制 Gist, 使用了无损和有损两种编码机制对特定的特征图进行编码表示, 并在反向传播时进行解码。针对池化层之后的 ReLU 层的输出结果, Gist 用 1bit 的数据代替 32bit 的数据进行存储; 针对卷积层之后的 ReLU 层的输出结果, 利用其稀疏性, 使用特征映射的方式进行存储。这两种编码方式都不会降低精度, 但只能用于包含 ReLU 层的神经网络模型。针对神经网络部署在嵌入式系统上的场景, Han 等人^[41]提出了“深度压缩”的方法, 包含剪枝、量化和霍夫曼编码三种压缩方法。该方法首先通过只学习重要的连接来对神经网络进行剪枝, 将权值较小的连接进行修剪; 接着是对参数进行量化, 用较少的比特数代表权值, 并进行权值共享, 在减少内存空间占用的同时保证不会降低精度; 最后, 使用霍夫曼编码将模型进行压缩。

此外, 在内存交换方法中, 由于 PCIe 总线带宽的限制, 数据进行传输的时间往往会大于计算的时间, 从而产生额外的时间开销。Rhu 等人^[42]提出了压缩直接内存访问引擎(compressing Direct-Memory-Access, cDMA), 将选择进行内存交换的数据进行压缩, 从而减少数据通过 PCIe 传输的时间, 进一步减少对于 GPU 内存的占用。由于神经网络中的 ReLU 层的输出数据具有稀疏性和高可压缩性, 该方法选择对 ReLU 层的输出数据进行压缩。由于卷积神经网络(CNN)中大量使用了 ReLU 层, 因此 cDMA 常用于 CNN 模型的压缩优化, 而在其他神经网络模型中优化效果不明显。

压缩参数可以精简网络结构,但剪枝、量化等压缩技术依赖于模型预训练,需要为预训练模型预留额外的内存空间。此外,压缩参数会带来精确度下降的问题,需要进行微调提升网络性能。压缩结构不需要预训练和微调,存储需求小且计算量低,但该方法构造的特殊结构泛化性差,同时无法进一步使用其他压缩方法提高网络性能。

1.3 研究目标和主要工作

本文的研究目标是针对 GPU 内存不足的问题,评估重计算方法和内存交换方法的性能开销,对不同的张量选择不同的存储和访问方式,为大批量的模型训练提供内存不足时的最小性能开销解决方案,减少深度神经网络训练过程中对于显存的占用,提高计算和存储设备的利用率,增大一次迭代处理的样本数量,减少整体训练时间。

本文的主要工作如下:

(1) 分析了张量在不同时间点进行内存交换对 DNN 训练过程的影响,以及选择不同张量进行重计算的时间开销变化,提出了精细粒度张量选择方法,根据 DNN 训练所需的内存空间大小和模型的计算数据流图选择张量进行内存交换或重计算,并确定了换入和换出张量的时间点。此方法对基于层的粗粒度选择方法进行优化,提升了 GPU 内存的利用率和 DNN 训练的速度,避免了粗粒度选择方法带来的额外时间开销。

(2) 在精细粒度张量选择方法的基础上,分析了训练时张量在主存与 GPU 内存之间的数据移动情况,以及内存空间占用动态变化情况,提出了动态张量选择方法,当换出张量无法减少内存空间占用时,根据内存峰值的变化修改部分内存交换张量为重计算张量,进一步减少训练过程中的内存空间占用。针对内存交换张量延迟释放的问题,提出了多阶段张量选择方法,保证决策结果的正确性。

(3) 通过对内存交换方法实施过程中内存分配和释放顺序的观察和分析,发现了内存交换方法带来的内存碎片问题,提出了张量放置策略,将频繁分配与释放的内存交换张量与保存在 GPU 内存中的张量分开放置,减少内存碎片发生的概率,从而降低内存碎片带来的额外内存空间占用。

1.4 全文结构

全文的组织结构如下：

第一章介绍了研究背景和国内外相关研究现状，阐述了研究目标和主要工作，包括深度神经网络场景下 GPU 内存不足的问题，现有使用内存交换和重计算方法的工作的问题。

第二章阐述了相关的研究和关键技术，包括深度神经网络的结构和训练过程，内存交换技术和重计算技术的工作原理及其适用场景和优缺点，以及现有训练框架对于 GPU 内存中碎片问题的解决方法。

第三章阐述了精细粒度动态内存管理方法 FDNN 的具体设计与实现，包括总体设计和各个模块的设计与实现，主要包括精细粒度张量选择方法、动态张量选择方法和张量放置策略。

第四章测试了 FDNN 动态决策过程中的决策结果变化，分析了不同决策结果的内存空间占用变化。对训练能达到的最大批处理大小和训练速度进行了测试和分析，与 vDNN、IPABN 和 SuperNeurons 工作进行对比，并且对测试结果给出了说明和分析。

第五章对全文进行总结，阐述了主要工作并分析工作的不足及需要改进的内容。

2 研究基础与相关技术

本章介绍了相关研究的基本概念和技术，包括深度神经网络训练模型、内存交换技术及重计算技术。进一步阐述了内存交换技术带来的内存碎片问题。

2.1 深度神经网络训练模型

深度神经网络被广泛用于许多机器学习任务，例如计算机视觉，自然语言处理，物体检测等。图 2-1 展示了 DNN 训练中对于一张图片的处理过程，DNN 训练中对图片划分区域提取特征，通过多个简单数学函数对特征进行计算处理，所提取的特征也由最基本的点、线、面变成了事物的具象化特征，例如车的轮胎，狗的尾巴等，最终依据这些特征给出不同标签的概率。神经网络层分为三类：输入层、输出层和隐藏层。一般来说第一层是输入层，最后一层是输出层，而中间有多个隐藏层。各种类型的层被用于不同的目的和应用，常用的层有卷积层(Convolutional Layer)、图像分类中的激活层(Activation Layer)、池化层(Pooling Layer)和自然语言处理中的注意力层(Attention Layer)等。每层的输入和输出即特征图，特征图共有 N , H , W 和 C 四个维度，分别表示一次迭代参与训练的图像数量，输入图片的高度，输入图片的宽度和输入图片的通道数，在深度神经网络的工作中，常使用张量(多维数组)表示特征图和梯度图。随着深度学习应用的增多，DNN 模型也在快速且持续地发展，不同类型的层可以通过各种方式进行连接，以寻求适合于各种问题的神经网络结构。

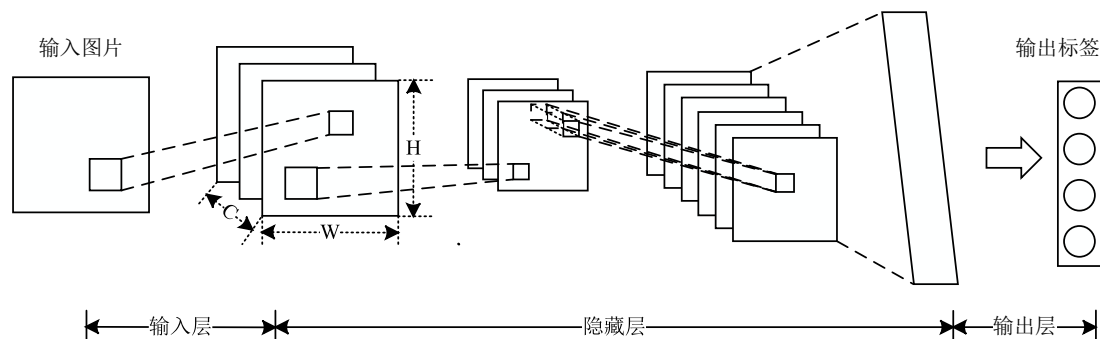


图 2-1 DNN 训练过程

DNN 训练的目标是找到一套合适的模型参数，满足一个目标函数，获得最理想化的模型，例如在图像分类的应用中，DNN 训练的目标是设置合适的权重以同时最

大化正确分类的分值和最小化其他不正确分类的分值，其中理想的正确分值与 DNN 基于当前权重计算的分值之间的差被称为损失值。为了获得理想化模型，需要找到一组能够最小化数据集中的平均损失值的权重。一次训练过程通常由数百万次迭代组成，每次迭代由两个计算阶段组成，即前向传播和反向传播。在前向传播过程中，输入层的输入为样本，隐藏层和输出层的输入为上一层的输出特征图，每层根据输入、权重和偏置计算输出特征图，这个过程不断重复直到最后一层。计算前向传播的输出层得到的预测值与真实标签的损失值。反向传播从输出层开始，输出层的输入为损失值，隐藏层和输入层的输入为上一层的输出梯度图，每层根据输入和前向传播中对应层的特征值计算输出梯度图，此外，每层的权重还会根据梯度图进行更新。现在有很多用于反向传播的优化器，包括随机梯度下降(Stochastic Gradient Descent, SGD)^[43], Momentum^[44]和 Adam^[45]等，用于找出一组最小化损失值的权重，从而进行更精准的预测。

由于前向传播产生的特征图会用于反向传播的计算，因此需要将特征图保存在 GPU 内存中。当网络层数较少时，保存前向传播产生的特征图不会造成内存容量不足的问题，但随着网络层数的增多，缓存特征图所需要的内存容量也增多。减少一次迭代处理的样本数量(即批处理大小)可以缓解内存容量不足的问题，但批处理大小太小会使得每次迭代产生的梯度相差较大，导致较大的训练结果误差。

当单个 GPU 内存不足时，可以对神经网络^{[46][47]}或者输入样本^{[48][49]}进行拆分并在多个 GPU 上并行训练。但事实上，由于特征图不仅内存占比高且为低频率访问数据，产生长时间占用内存空间而不被访问的情况，而 GPU 价格昂贵且用于并行高速的数据计算，如果将所有特征图保存到多个容量有限的 GPU 内存中，会造成资源的浪费。

2.2 支持 DNN 的内存交换技术

为了解决存储器件写入速度慢与处理器高速计算之间的性能差别问题，内存交换技术被广泛地应用于计算机系统中，这也适用于解决深度神经网络训练场景中的 GPU 内存不足的问题。

2.2.1 内存交换技术的工作原理

内存交换技术最早被用于解决计算机的物理内存有限的问题^[50]。在操作系统中,内存交换技术的核心解决思路是,当系统的物理内存不足时,不常访问的数据保存到磁盘中,而当系统访问磁盘中存储的内容时,再将磁盘上的数据加载到内存中,这两种操作就是内存交换技术中的换出和换入。内存交换技术可以避免由于内存不足导致的进程崩溃等问题,也可以通过将常用的数据移动到靠近处理器的存储空间中以提高系统的整体性能。

一个程序的数据访问模式对内存交换技术带来的收益有显著影响。当一个程序访问的数据呈现较好的空间局部性时,内存交换技术可以带来显著的性能提升,但对于空间局部性差的程序,内存交换技术会带来频繁的数据移动,从而导致系统整体性能下降。

2.2.2 基于内存交换的深度神经网络

在深度神经网络训练的场景中,内存交换技术是指在 GPU 内存和主存之间交换数据,将长时间不访问的数据换出到主存中,并在访问已换出数据时再换入到 GPU 内存中,从而减少对于 GPU 内存的最大占用,提高 GPU 内存的利用率。深度神经网络呈现一种规律的数据重用模式,对于数据的读写有一定规律,空间局部性高,因此内存交换技术可以很好地应用于深度神经网络。在理想情况下, GPU 内存和主存之间的数据通信开销可以被计算操作隐藏,带来很少的额外时间开销。Rhu 等人^[23]提出了基于内存交换的内存管理方案,称之为虚拟深度神经网络(virtualized Deep Neural Network, vDNN),该工作将 GPU 内存和主存看作支持训练 DNN 的共享虚拟内存。在前向传播时, vDNN 将部分层计算的中间数据换出到主存中,减少了 GPU 的内存空间占用。在反向传播时, vDNN 在适当的时候将这些中间数据换入到 GPU 内存中,由于反向传播过程中每完成一层计算都会有部分数据的生命周期结束,其占用的内存空间会被释放,因此换入数据不会造成进一步的内存消耗。内存交换技术带来的性能收益在于隐藏通信开销,但 vDNN 在每一层结束时会将数据传输和计算同步,因此只有当计算时间足够长,数据传输开销才能被很好地隐藏。由于深度神经网络

络中卷积层比激活层和池化层等计算时间长，vDNN 选择将卷积层的输入数据换出到主存，从而减少由于内存交换带来的额外时间开销。

内存交换方法受 PCIe 总线带宽的影响，能隐藏的数据通信开销有限，因此内存交换决策的关注点是如何选择合适的时间点进行内存交换，从而将数据通信时间和计算时间重叠最大化，减少内存交换方法带来的额外时间开销。

2.3 支持 DNN 的重计算技术

重计算技术的核心思想是利用空闲的计算资源来减少对于内存的占用，常用于内存资源紧缺而计算资源充足的场景下，也适用于深度神经网络的大批量训练。

2.3.1 重计算技术的工作原理

重计算技术最早由 Kandemir 等人^[51]提出，用于解决嵌入式等应用场景下内存空间不足的问题。此工作的核心思想是“替代”，具体来说重新计算一个代码块的结果，替代将结果存储在内存中并在需要时执行内存访问的操作。在具体实施过程中，根据数据流图选择需要重计算的代码块。数据流图中每个节点代表代码块的计算结果，边代表这些计算结果的数据依赖关系，箭头的指向即代表前者节点的输出是后者节点的输入。当某节点被决策为重计算节点，则在执行过程中，每当需要访问该节点计算的结果时都重新计算出该数据，而被决策为存储节点的数据则为其分配存储空间，并保留该分配直至执行结束。

此工作在有复杂数据依赖关系和大量数据重用的程序中有明显的性能提升，但对于数据结构简单，没有太多数据重用的场景下，该方法的收益并不明显。

2.3.2 基于重计算的深度神经网络

由于深度神经网络呈现一种规律的数据重用模式，且层数越深，神经网络的数据依赖关系越复杂，重计算技术能够有效支持深度神经网络训练场景。Samuel 等人^[52]提出了一种系统的重计算方法 InPlace-ABN，用于减少深度神经网络训练过程中对于内存空间的占用。具体来说，因为深度神经网络在前向传播过程中会保存每一层计算产生的特征图，并在反向传播时利用保留的特征图信息计算梯度，重计算方法则是在

前向传播过程中将部分特征图丢弃，在反向传播期间，被丢弃的特征图被访问时，利用没有丢弃的特征图信息重新计算出该特征图。

重计算方法的研究难点是估算每个特征图的重计算开销。因为特征图的重计算需要根据其他特征图进行计算，当保存的特征图数量较少时，重计算其他特征图的时间开销会较大。当保存的特征图数量较多时，对于内存空间占用的减少量不显著。此工作对如何选取重计算张量进行了进一步地探讨，选择计算时间较短的层计算产生的张量进行重计算，例如批标准化层和激活层等。

面向深度神经网络的重计算技术分为静态和动态两种，静态重计算方法在训练前对所有张量进行决策，使用 0-1 变量表示某张量被访问时是执行重计算操作还是从内存中读取。动态重计算方法则根据训练过程中张量的生命周期和 GPU 内存空间占用情况对决策方案进行动态调整，即在内存容量充足时存储数据，而在不足时将数据丢弃，并在其再次被访问时重新计算出来。静态方法无法得到最佳的决策方法，但易于实现，且不会在执行期间产生额外的决策时间开销。动态方法能很好地适应存储器空间占用的动态变化，可以最大限度地利用有限的内存空间。值得注意的是，由于一次深度神经网络的训练过程通常由数百万次的迭代组成，即使一次迭代过程中动态方法产生的额外时间开销很小，在此场景下仍然会产生较大的额外时间开销。

重计算方法可以解决深度神经网络训练过程中 GPU 内存不足的问题，但往往 GPU 内存不足时，GPU 的计算资源也耗尽，这说明重计算方法一定会产生额外的时间开销用于重新计算，因此重计算决策的重点是选择时间开销小而内存空间占用量大的张量。

2.4 DNN 训练中的内存碎片问题研究

在操作系统的内存分配过程中，使用连续分配算法来管理内存分配时会产生内存碎片，主要分为内部碎片和外部碎片两种情况^[53]。内部碎片是指所分配的内存空间大于请求所需的内存空间时，部分空间空闲但无法被回收利用的情况；外部碎片是指还没有被分配出去的内存空间，但因其容量太小而无法分配给申请的新进程而产生的空闲区域。

以 Pytorch 训练框架^[54]中的内存管理机制为例,介绍现有的框架针对内存碎片问题的解决方案。为了减少频繁的内存分配和释放,Pytorch 的显存管理机制会预先分配一个连续的内存块作为内存池,而管理内存的基本单位是一个可变容量的块,所有地址连续的块的元数据都被存储在一个双向链表中。对于一个内存分配请求,系统会在链表中找到足够大的空闲内存块,并返回其内存地址,但往往该内存块的大小会大于请求所需要的内存空间,因此会造成内部碎片的问题。此时分配器会检查被分配块中未被使用的内存空间的大小,如果大于一定的内存阈值,例如 512 Bytes 或者 1 MB,则会对此内存块进行拆分,从而缓解内部碎片的问题。设定内存阈值则是为了避免拆分出较小的内存块,使得该内存很难被利用,产生外部碎片。当内存块被释放时,会检查其前后内存块是否为空,若为空则会将相邻的块进行合并(不为空则直接释放),从而减少外部碎片。

在神经网络训练的场景下,由于其规律的数据重用模式,前向传播时生成的张量会被分配一段连续的地址空间,而在反向传播时会按照张量生成的顺序反向释放内存空间,因此几乎不会产生内存碎片,例如 Pytorch 等训练框架也给出了内存碎片的解决办法,但随着神经网络模型的发展,出现了一些结构较为复杂的非线性模型,会进行乱序的内存分配和释放,产生一定的内存碎片。常用于 GPU 内存管理的内存交换技术也会导致部分内存块的提前释放,使得外部碎片增加,具体的问题描述会在 3.3 节中阐述。

2.5 本章小结

本章节介绍了和本文研究内容相关的主要技术和原理。首先介绍了深度神经网络的训练过程。其次介绍了内存交换和重计算的原理,两者在神经网络中的应用,探讨了其优缺点和研究难点。最后介绍了内存碎片问题的概念、现有的显存管理机制中解决内存碎片问题的技术方法、以及神经网络训练场景下产生内存碎片的原因。

3 精细粒度的动态内存管理方法设计与实现

本章针对粗粒度内存管理方法的问题，提出了面向深度神经网络训练的精细粒度动态内存管理方法 FDNN (Fine-grained Dynamic Memory Management for Deep Neural Network)。共有三个设计点：精细粒度张量选择方法、动态张量选择方法和张量放置策略。第一个设计分析了重计算和内存交换方法的性能开销，在任意内存需求下都能给出最小性能开销的张量选择方案。第二个设计分析了不同决策的内存空间占用的动态变化，在第一个设计的基础上给出了动态张量选择方法。第三个设计分析了内存交换方法带来的内存碎片问题，在前两个设计的基础上提出了张量放置策略。

3.1 总体设计

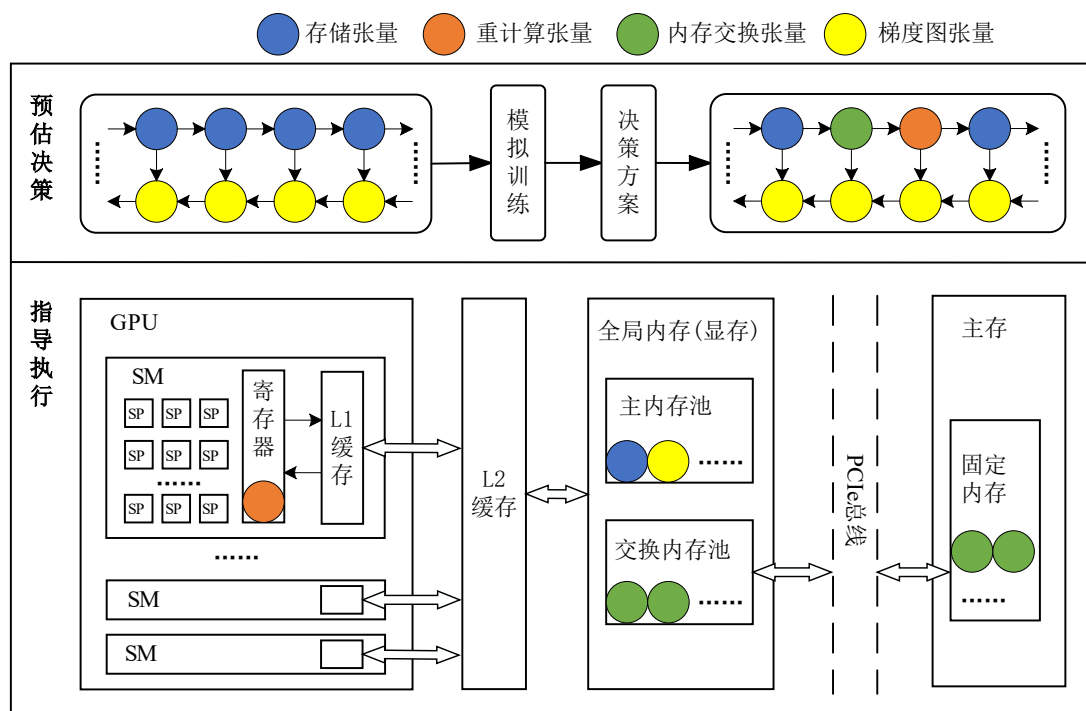


图 3-1 精细粒度的动态内存管理方法设计框架

图 3-1 展示了精细粒度的动态内存管理方法设计框架，分为预估决策和指导执行。

(1) 预估决策。

针对 1.1 节中提到的基于层的粗粒度内存管理方法的问题, 3.2 节量化了内存交换和重计算的性能开销, 提出了精细粒度张量选择方法, 在不同内存需求下给出性能开销最小的决策方案。3.3 节分析了不同选择方案的内存空间占用动态变化情况, 提出了动态张量选择方法, 保证了决策的正确性。

由于张量大小和每层的计算时间会随着批处理大小变化, 因此在决策前需进行模拟训练得到以上信息用于张量的选择与决策。模拟训练即执行十次迭代, 测量记录所有张量的大小和每层计算时间的平均值, 为了避免模拟训练时产生内存不足的情况, 将所有生成的张量都转移到主存中, 任意时刻 GPU 内存中都只保存了当前层的输入、输出和卷积工作区。

根据模拟训练得到的信息以及 DNN 模型的训练数据流图进行决策, 可以得到更新后的数据流图, 更新的内容即对前向传播中计算产生的特征图张量打上不同的标签。图中的绿色节点代表内存交换张量, 橙色节点代表重计算张量, 蓝色节点代表存储张量, 黄色节点代表梯度图张量, 在指导执行的过程中会依据不同的标签对张量进行不同的存储和访问操作。

(2) 指导执行。

图 3-1 展示了 GPU 的存储架构。一个 GPU 中有多个 SM(Streaming Multiprocessor, 流多处理器), 每个 SM 中包含多个 SP(Streaming Processor, 流处理器)和一些存储资源(寄存器、L1 缓存等)。SP 是 GPU 中最基本的处理单元, 一个 SP 对应一个线程, 因此一个 SM 可以实现多个线程的并行执行。

SM 中的寄存器是 GPU 中访问速度最快的存储资源, 重新计算出的张量会存放在寄存器中, 直接用于当前层的计算, 由于长期将数据存储在寄存器中会影响 GPU 的计算效率, 反向传播中会在访问重计算张量时才开始进行重新计算。存储张量和内存交换张量都存放在显存中(GPU 内存, 后文中的内存均指显存)。由于内存交换将张量换出到主存中后提早释放了其占用的 GPU 内存空间, 打乱了原本张量分配与释放顺序, 可能造成内存碎片问题。为了避免碎片问题, 在显存中为张量分配了两块内存池, 主内存池中存放存储张量和梯度图张量, 交换内存池中存放内存交换张量, 3.4 节阐述了内存碎片产生的原因, 并提出了张量放置策略解决该问题。内存交换张量通

过 PCIe 总线复制到主存中提前分配的一块固定内存(Pinned Memory)中,操作系统不会对这块固定内存空间进行分页和交换操作, GPU 可以通过直接内存访问(Direct Memory Access, DMA)机制,在 GPU 内存和主存之间快速地复制数据。为了避免访问内存交换张量时该张量还未从主存中复制到 GPU 内存中,会在访问该张量前对复制操作进行同步,等待复制操作完成后再进行接下来的访问计算。3.5 节阐述了精细粒度的动态内存管理方法的具体实现。

3.2 精细粒度张量选择方法

现有的工作通常对同一类型的层计算产生的张量进行相同的决策,例如 SuperNeurons 选择卷积层的输入张量进行内存交换,选择池化层、激活层和批标准化层的输入张量进行重计算。但同种类型的层的计算时间和张量大小受很多因素的影响,粗粒度的决策并不能提供性能开销最低的决策方案,限制了内存的优化空间。本节分析了不同决策方案的内存交换开销和重计算开销,提出了内存交换和重计算权衡策略,对 DNN 训练的任意内存需求都提供了性能开销最小的基于张量的细粒度决策方案。

精细粒度张量选择方法主要解决的问题包括:

- (1) 量化内存交换和重计算方法的开销;
- (2) 确定换入和重新生成张量的时间点;
- (3) 确定优先逐出(即释放该张量占用的内存空间)的张量;
- (4) 内存交换和重计算方法的选择。

3.2.1 和 3.2.2 节主要讨论第一和第二个问题的解决方法,3.2.3 节主要讨论第三和第四个问题的解决方法。

3.2.1 内存交换开销分析

本小节讨论了内存交换张量及其换入换出时间点的选择,目标是使数据传输的时间开销尽可能被计算时间覆盖,减少内存交换方法带来的额外时间开销。

数据在主存与显存之间的传输通常采用与计算异步的方式,因此进行内存交换时,数据传输的部分时间开销会被计算时间开销覆盖。如果数据传输时间小于计算时

间,进行内存交换不会产生额外的时间开销,如果数据传输的时间大于计算时间,内存交换方法会带来额外时间开销,其大小为数据传输时间与计算时间之差。

在部分使用内存交换的工作中(如 vDNN),在每层计算结束后会对计算和数据传输进行同步,即在每一层计算完成后,不立即进行下一层的计算,而是等待当前层的输入数据换出完成后再进行下一层的计算。如图 3-2 所示,绿色方块代表内存交换时间,蓝色方块和黄色方块分别代表前向传播和反向传播某层的计算时间。由于 PCIe 的带宽有限,张量的内存交换时间往往大于当前层的计算时间,因此该同步操作会使得每个张量的换入换出都产生一定的额外时间开销。

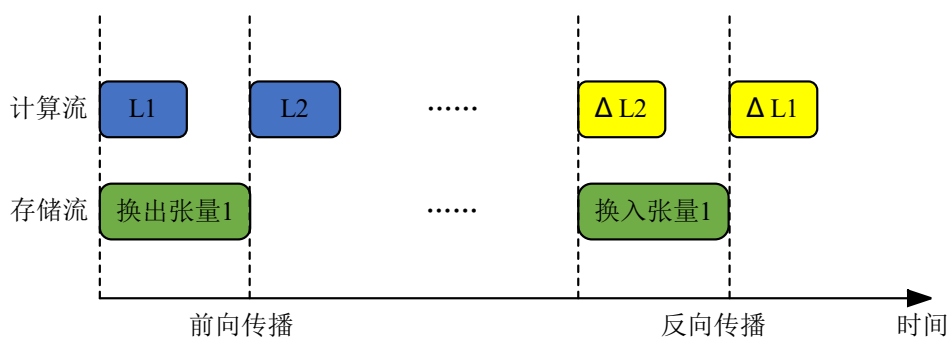


图 3-2 计算和数据传输同步的情况

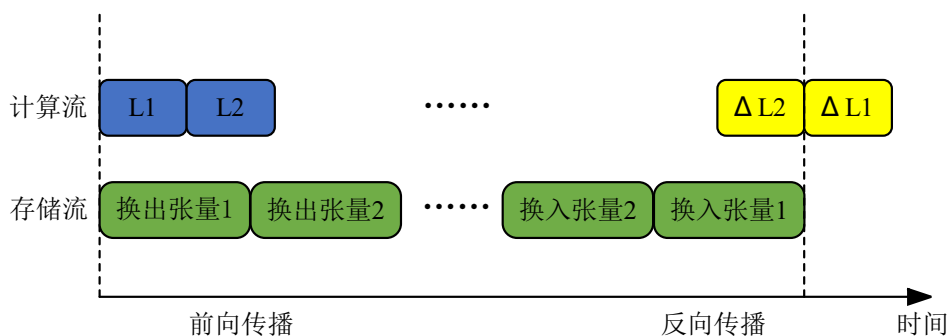


图 3-3 计算和数据传输不同步的情况

取消同步操作可以有效地减少额外时间开销,如图 3-3 所示,如果不等待当前层的输入数据换出完成,直接进行下一层的计算,可以使内存交换时间与计算时间更多地重叠,减少内存交换方法带来的额外时间开销。

进行同步的好处是,张量会在下一层开始计算前交换到主存中或 GPU 内存中,使得训练过程始终保持最小的内存空间占用,例如,图 3-2 中第二层进行计算时,张量 1 已经完成换出,因此可以释放其内存空间,此时内存空间占用只包含当前层的

输入、输出和卷积工作区。而图 3-3 中，第二层进行计算时，张量 1 未完成换出，因此无法释放其内存空间，此时内存空间占用还包含张量 1。但事实上，训练过程中内存空间占用的峰值往往位于反向传播的起始阶段，因此对于内存空间占用不是峰值的层来说，增加一些内存空间占用对于内存交换方法的性能优化结果的影响较小。因此，FDNN 采用计算和数据传输不同步的内存交换方法，减少因同步带来的额外时间开销。

值得注意的是，不同步的收益仅限于交换少量数据，当需要进行交换的数据较多时，计算时间没有办法覆盖内存交换时间，依然会产生额外的时间开销，3.2.3 节给出了这种情况下的张量选择方法。此外，由于数据不能及时地换出，可能会造成内存碎片问题，降低 GPU 内存的利用率，在 3.4 节详细介绍了此问题，并给出了解决方案。

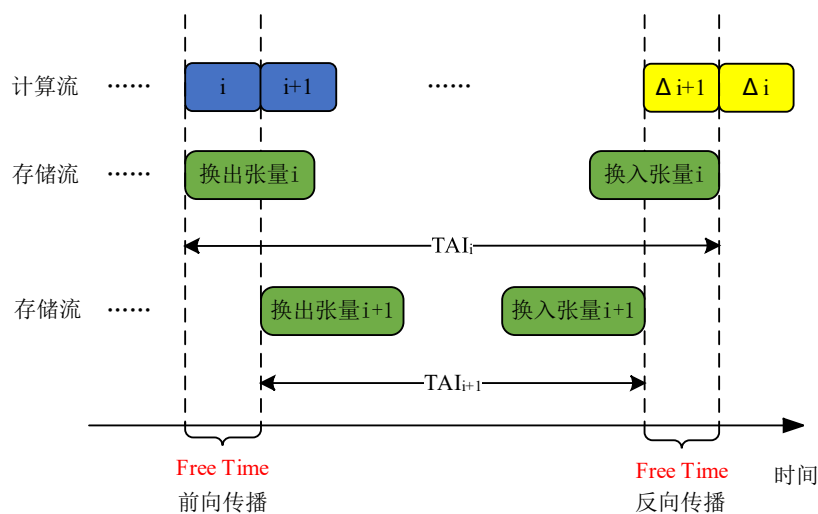


图 3-4 选择 TAI 不同的张量进行内存交换

张量访问时间间隔(Tensor Access Interval, TAI)越大，进行内存交换后可以尽早释放对于 PCIe 资源的占用，选择 TAI 大的张量进行内存交换可以使内存交换时间和计算时间更好地重叠。由于张量的计算时间开销是一定的，张量访问时间间隔可以根据模拟训练时获得的信息进行计算。如图 3-4 所示， TAI_i 即为前向传播时第 i 层开始计算的时间点(张量 i 第一次被访问)与反向传播时第 i 层开始计算的时间点(张量 i 第二次被访问)的时间间隔。图中展示了一次训练过程中，第 i 层和第 $i+1$ 层的计算时间线和换出不同张量的存储时间线，并假设 PCIe 总线资源在第 i 层开始计算时被释放。

如果选择 TAI 较小的张量 $i+1$ 进行内存交换, PCIe 总线会空闲一段时间, 即图中的 FreeTime, 且由于选择 $i+1$ 以后的张量进行内存交换时, 只能等待张量 $i+1$ 换出结束后再进行换出操作, 后续操作无法填充 FreeTime, 使得在 FreeTime 时间段中, 内存交换时间没有与计算时间进行重叠, 损失了内存交换方法本可以带来的性能提升。因此, 本设计优先选择 TAI 较高的张量进行内存交换。

由于张量换出的时间大于张量计算的时间, 有时甚至可以达到计算时间的数倍, 因此在一个张量换出完成并释放 PCIe 资源后, 可能有多个可供选择进行内存交换的张量, 如图 3-5 所示, 假设张量 $i-1$ 换出完成后, 恰好第 i 层完成计算, 第 $i+1$ 层开始计算, 那么此时可供选择进行内存交换的张量有张量 i 和张量 $i+1$, 两者的区别在于张量大小不同。本文优先选择较小的张量进行内存交换, 这是由于选择较大的张量进行内存交换, 可能大大超过了内存需求量(正常模型训练占用的内存与内存阈值的差), 反而带来不必要的时间开销。

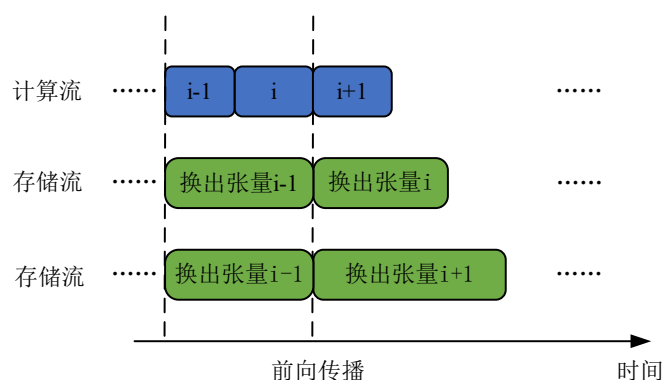


图 3-5 选择不同大小的张量进行内存交换

根据以上分析, 优先选择 TAI 高而计算时间小的张量进行内存交换, 因此本设计引入了张量交换时间间隔(Tensor Swap Interval, TSI)这一指标, 作为内存交换方法的选择标准。TSI 的计算方法如公式(3-1)所示, TAI 表示张量访问时间间隔, 而 SwapTime 为张量进行内存交换所需的时间。

$$TSI = TAI - SwapTime \quad (3-1)$$

但观察图 3-5 的计算和交换时间线, 发现选择张量 $i-1$ 进行内存交换后, 后续张量(例如张量 i)需要等待张量 $i-1$ 换出完成后才能换出, 并不是所有张量都从对应层开始计算的时间点进行换出, 因此并不能完全根据 TAI 进行决策, 还需考虑上一个

决策后剩余的空闲时间，即上一个决策得到的 TSI，因此 TSI 最终的计算方法如公式 (3-2)所示， TSI_k 表示第 k 次决策计算得出的 TSI， TAI_i 表示张量 i 的 TAI。

$$TSI_{k+1} = \min(TSI_k, TAI_i) - SwapTime \quad (3-2)$$

图 3-6 进一步展示说明了 TSI 的计算过程，第 k 次决策时选择了张量 $i-1$ 进行内存交换， TSI_k 为张量 $i-1$ 换出完成到开始换入的时间。进行第 $k+1$ 决策时，由于张量 i 和 $i+1$ 的 TAI 分别大于和小于 TSI_k ，因此，在计算张量 i 的 $TSI_{k+1,i}$ 时根据的是 TSI_k ，而在计算张量 $i+1$ 的 $TSI_{k+1,i+1}$ 时根据的是 TAI_{i+1} 。由于 $TSI_{k+1,i}$ 大于 $TSI_{k+1,i+1}$ ，优先选择 TSI 较大的张量进行内存交换，优先选择张量 i 进行内存交换，即第 $k+1$ 次决策产生的 TSI_{k+1} 为 $TSI_{k+1,i}$ ，之后的决策又基于 TSI_{k+1} ，如此循环往复，直到内存交换时间无法被计算时间覆盖。

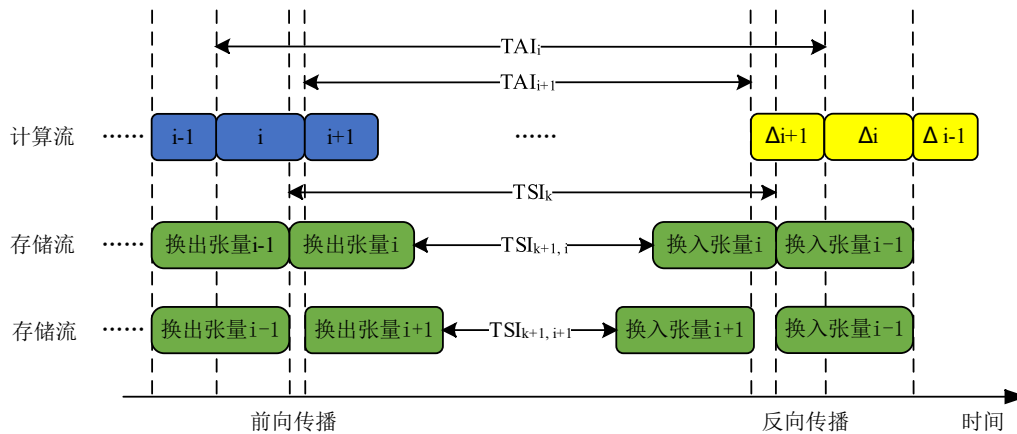


图 3-6 TSI 计算示意图

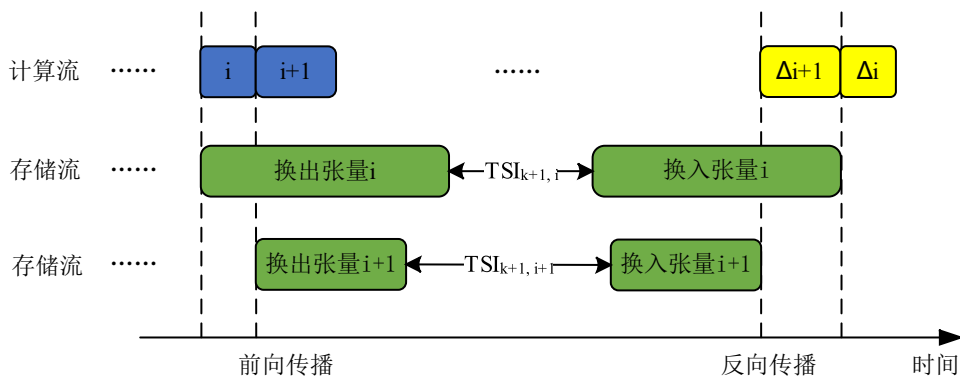


图 3-7 特殊情况示意图

张量 i 的 TAI 较大，内存交换时间较长，而张量 $i+1$ 的 TAI 较小，内存交换时间

较短时，依据 TSI 并不能选择开销最小的张量。如图 3-7 所示，根据 $TSI_{k+1,i}$ 小于 $TSI_{k+1,i+1}$ 的计算结果，按照优先选择 TSI 较大的张量的原则，应选择张量 $i+1$ 进行内存交换，但选择换出张量 $i+1$ 时，会出现部分计算时间无法被覆盖的情况，这与图 3-4 的情况类似。针对这种情况，会根据内存阈值进一步选择，如果换出张量 $i+1$ 后，DNN 训练所需的内存容量需求已经得到满足，那么选择张量 $i+1$ 进行内存交换，否则选择张量 i ，保证尽量完全覆盖计算时间。

内存交换时间开销 $SwapTime$ 分为两部分：1)前向传播时，将张量换出到主存的时间；2)反向传播时，从主存中预取张量的时间。事实上，这两部分的时间开销基本相同，且与张量大小成正比，因此计算内存交换时间开销时只需计算其中一部分的时间开销即可。内存交换时间的计算公式(3-3)所示， $SwapTime$ 为张量进行内存交换所需的时间， T_Size 表示张量大小， $PCIeBW$ 表示 PCIe 总线通信带宽。

$$SwapTime = 2 * \frac{T_Size}{PCIeBW} \quad (3-3)$$

由于 PCIe 的带宽有限，张量的传输时间能被计算时间覆盖的部分有限，当传输时间无法被计算时间覆盖时，则按需选择张量进行内存交换。对于一定的内存需求量，有多种张量的组合选择可以满足内存需求量，由于内存交换时间与释放内存大小成正比，释放的内存越大，内存交换需要的时间越长，因此，选择张量大小之和最小的组合即可保证产生的额外时间开销最小化。

3.2.2 重计算开销分析

重计算方法在前向传播时释放部分张量的内存空间，在反向传播访问该张量时利用 GPU 计算资源将其重新计算出来，降低了整体训练过程的内存空间占用。当 GPU 的内存空间不足时，GPU 的计算资源也会用尽，因此重计算方法一定会产生额外的时间开销。在使用重计算方法时，优先选择重计算时间小的张量，这可以有效减少额外的时间开销。需要注意的是，重计算方法采取按需的方式，在访问重计算张量时才进行重计算，避免过早地生成重计算张量，需将重计算张量存放在 GPU 内存中。这样可以避免在内存使用高峰期重新计算生成的张量占用过多内存空间导致内存不足。

张量的重计算时间主要取决于深度神经网络模型的结构和批处理数据大小，这部分时间开销很难通过模型的基础信息进行计算，需要在模拟训练阶段对每层的计算时间进行记录，再根据模型的数据流图计算每个张量的重计算时间。在本节对于重计算时间开销的讨论中，每个张量的计算时间被视为已知量。

由于每层计算产生的张量大小不同，存储开销也不同，仅根据重计算时间选择张量进行逐出，可能无法获得性能开销最小的方案。以批处理大小为 256 的 AlexNet 模型训练的某一次迭代过程为例，如图 3-8 所示，每一层的计算时间和计算产生的张量大小都不同，且两者的大小并没有呈现明显的关联。假设需要释放 200 MB 的内存，如果仅按照计算时间从小到大选择张量，则其选择的顺序为张量 7 和张量 3，一共释放了 224.5 MB 内存，产生了 5.9 ms 额外时间开销。但通过观察发现，只选择张量 1 进行重计算，释放了 283.6 MB 内存，只产生了 4.1 ms 额外时间开销，此选择方案明显优于仅根据计算时间大小做出的选择。

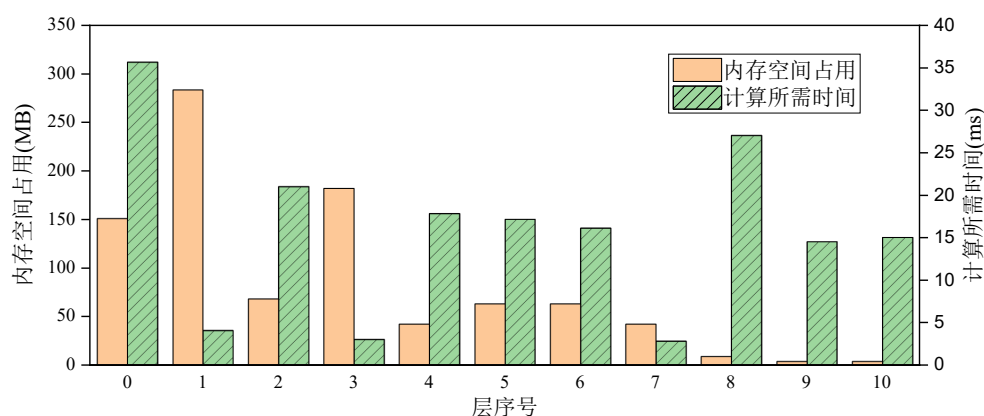


图 3-8 AlexNet 每层计算时间和生成张量的内存空间占用

基于以上观察，在选择重计算张量时，选择释放单位内存空间所需时间最小的张量才能产生最小的性能开销。本工作引入了内存价值(Value For Memory, VFM)这一指标，用于衡量释放单位内存空间产生的额外计算时间，并作为重计算方法的选择标准，张量的 VFM 越小，越适合进行重计算。VFM 的计算方法如公式(3-4)所示，GenerationTime 表示重新生成张量的时间开销，T_Size 表示张量大小，即逐出该张量所释放的内存大小。

$$VFM = \frac{GenerationTime}{T_size} \quad (3-4)$$

张量的 $GenerationTime$ 初始值为对应层的计算时间，然而，张量的父节点张量并不会一直稳定地保存在 GPU 内存中，因此，在预估决策阶段，计算一个张量 A 的重计算时间 $GenerationTime$ 还需要考虑两种情况：

- (1) 张量 A 的父节点张量的生命周期是否结束，若结束了，则无法作为张量 A 的计算源；
- (2) 张量 A 的父节点张量是否在前向传播时被逐出(进行内存交换或重计算)，若被逐出，则无法作为张量 A 的计算源。

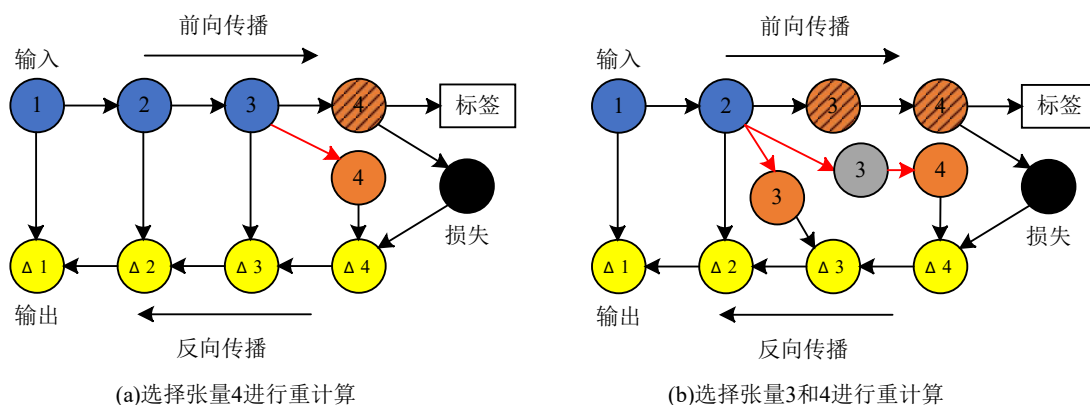


图 3-9 选择不同重计算张量的训练数据流图

用一个简单例子说明以上两种情况。假设一个线性模型训练中共生成四个张量，其中张量 4 为重计算张量，数据流图如图 3-9(a)所示，蓝色节点为前向传播过程中产生的特征图张量，黄色节点为前向传播过程中产生的梯度图张量，斜线橙色节点代表重计算张量，且在前向传播过程中被丢弃(即不保存在 GPU 内存中)，纯色橙色节点代表在反向传播过程中重新计算出来的张量。对于第一种情况，由于前向传播和反向传播对于特征图张量的访问顺序相反，张量 4 的父节点张量 3 在反向传播时，一定在张量 4 之后被访问，其生命周期也一定会在张量 4 之后结束，因此一般来说不会产生一个张量的父节点张量生命周期先于该张量结束的情况。对于第二种情况，假设张量 3 也为重计算张量，由于张量 3 在前向传播时被逐出且晚于张量 4 进行重计算，GPU 内存中没有保存张量 3，因此在反向传播阶段重计算张量 4 时，无法通过张量 3 计算张量 4，需要将张量 2 作为张量 4 的计算源，如图 3-9(b)所示。图 3-9(a)和图

3-9(b)中的红色箭头代表重计算的计算流程,比较两张图可以发现,当张量4的父节点张量3没有被逐出时,其 GenerationTime 为第三层的计算时间,当张量4的父节点张量3被逐出时,其 GenerationTime 为第二层和第三层的计算时间之和,重计算时间发生了改变,因此每选择一个张量成为内存交换或重计算张量,都需要对其他张量的 VFM 进行更新。

多个连续张量被选为重计算张量,会产生多次重复计算。以图 3-9(b)为例,在重新生成张量4的计算路径中,张量3已经被计算出来了,但并没有保存在 GPU 内存中(用灰色节点表示),而是在下一层计算需要张量3时再进行计算,这引入了更多的时间开销,但如果将计算路径中生成的张量全部保存下来,又会提前占用 GPU 的内存空间,有造成内存不足的风险。为了解决这个问题,本工作提出了重计算张量的动态选择方法,按照 VFM 从小到大选择张量进行重计算,当选择的某个张量的子节点张量为重计算张量,会将子节点张量重置为未被决策的张量并更新其 VFM。以图 3-9(b)为例,假设先选择张量4进行重计算,由于选择张量3会影响张量4的重计算路径和时间,影响决策的正确性,动态选择方法选择保留张量3为重计算张量的决策,将张量4重置为未被决策的张量并更新其 VFM,后续继续在未被决策的张量中选择 VFM 最小的张量进行重计算。

3.2.3 内存交换和重计算权衡策略

根据 3.2.1 节和 3.2.2 节的分析内容,本节提出了内存交换和重计算权衡策略,给出了张量逐出的选择顺序,并为逐出张量选择性能开销较小的逐出方法。

由于内存交换方法可以通过数据传输与计算并行隐藏其时间开销,而重计算方法一定会产生额外的时间开销,所以本设计优先选择内存交换方法,直到无法隐藏时间开销,再根据内存交换和重计算的性能收益进行混合决策。

选择逐出张量的流程图如图 3-10 所示,主要分为两个阶段:(1)内存交换选择阶段。选择合适的张量进行内存交换,使得计算开销尽量覆盖内存交换开销;(2)混合方法选择阶段。计算剩余所有张量的内存交换方法和重计算方法的时间开销,选择时间开销最小的张量及其对应方法。如果第一个阶段结束时,减少的内存空间占用已经足够完成模型训练,则提前结束张量的选择与决策,否则进入第二个阶段;如果逐出

了所有的张量仍然不能完成模型训练，则表示该模型某一层的输入输出和卷积工作区已经大于 GPU 提供的内存容量，只能降低批处理大小或在多个设备上并行训练。

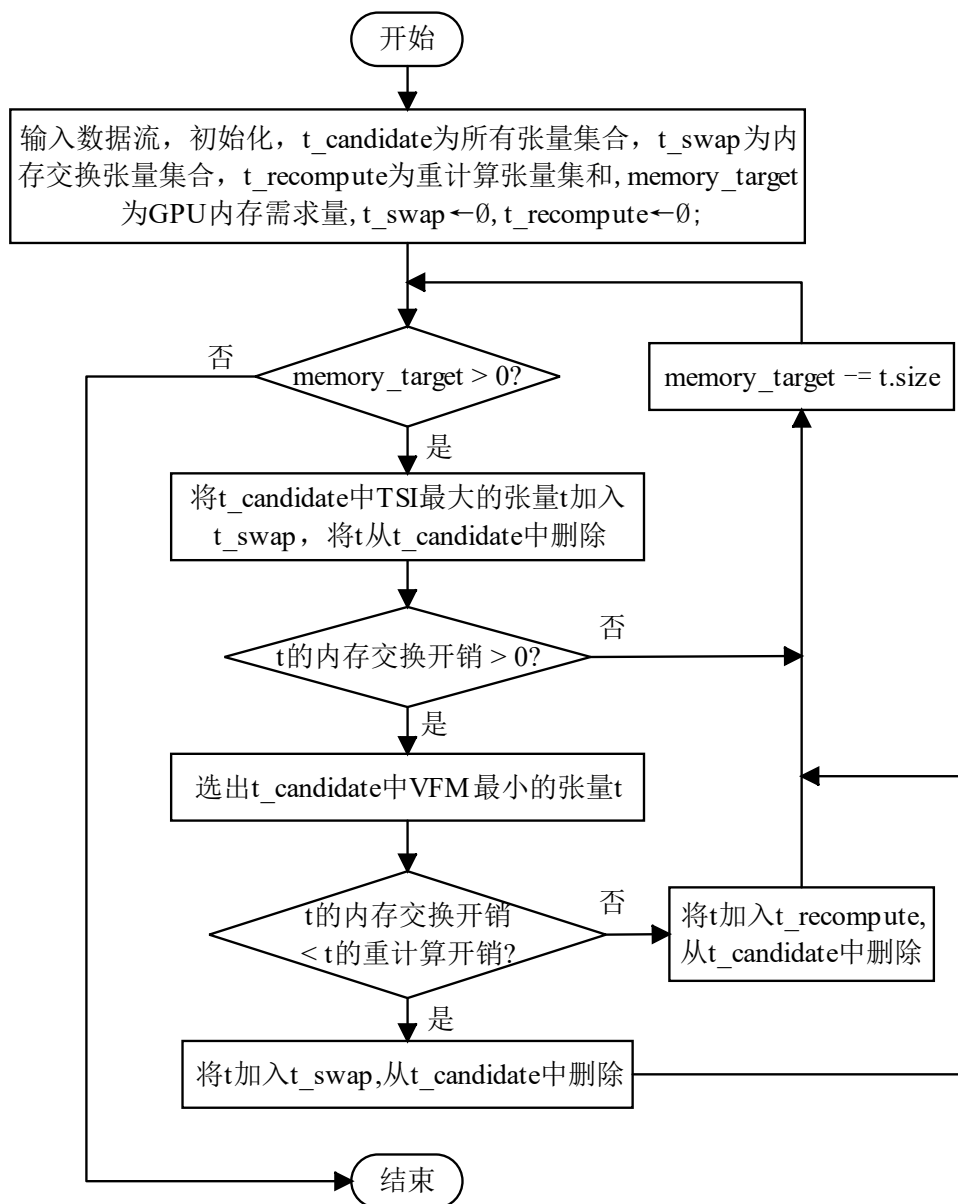


图 3-10 选择逐出张量的流程图

a) 内存交换选择阶段

内存交换选择阶段的内存交换张量选择算法如算法 3.1 所示，根据 TSI 值的大小在 $t_candidate$ 集合中选择合适的张量进行内存交换，将该张量插入 t_swap 集合，并从 $t_candidate$ 集合中删除，将内存交换张量的逐出和预取时间点插入 $swap_point$ 列

表。 $t_candidate$ 集合中存放未被决策的张量， t_swap 集合中存放内存交换张量， $swap_point$ 列表中存放内存交换张量的逐出和预取时间点。

算法 3.1: 内存交换选择算法

输入: 张量候选集合($t_candidate$), 目标内存需求量($memory_target$)

输出: 内存交换张量集合(t_swap), 内存交换时间点列表($swap_point$)

```
1.  $i \leftarrow 0, k \leftarrow 1, TSI_0 \leftarrow compute\_time;$ 
2.  $t\_TSI \leftarrow SortTSI(t\_candidate, TSI_0);$ 
3. while  $memory\_target > 0$  do
4.    $t \leftarrow t\_TSI_i;$ 
5.   while  $t.TAI < TSI_{k-1}$  do
6.     if  $memory\_target - t.size < 0$  then
7.       break;
8.     end
9.     else
10.       $t \leftarrow t\_TSI_{i++};$ 
11.    end
12.  end
13.   $TSI_k \leftarrow TSI_{k,t};$ 
14.  if  $TSI_k < 0$  then
15.    break;
16.  end
17.   $t\_swap.add(t);$ 
18.   $swap\_point \leftarrow \{t.EvitTime, t.PrefetchTime\};$ 
19.   $memory\_target \leftarrow memory\_target - t.size;$ 
20.   $t\_candidate.delete(t);$ 
21.   $UpdateRecompute(t\_candidate, t);$ 
22.   $t\_TSI \leftarrow SortTSI(t\_candidate, TSI_k);$ 
23.   $k++;$ 
24. end
```

算法 3.1 的工作步骤如下:

(1) 未开始决策时, TSI_0 为完成一次迭代的计算时间(即所有张量计算时间之和)。i 初始化为 0, k 初始化为 1。对 $t_candidate$ 集合中的所有张量的 TSI 从大到小排序, 得到张量序列 t_TSI 。

(2) 当 TSI 最大的张量 t_TSI_0 的 TAI 小于 TSI_{k-1} , 则按照 t_TSI 的顺序对未被决策的张量进行遍历, 若找到某个张量的大小大于内存需求量, 那么选择该张量进行内存交换, 否则选择 TAI 等于 TSI_{k-1} 的张量进行内存交换。被选择的张量记为 t 。

(3) 更新第 k 次决策的 TSI_k , 其值为张量 t 的 $TSI_{k,t}$ 。

(4) 当 TSI_k 小于 0, 说明内存交换时间无法完全被计算时间覆盖, 需要按照算法 3.3 进行进一步决策, 结束内存交换选择算法。

(5) 将张量 t 插入 t_swap 中, 从 $t_candidate$ 中删除该张量。插入张量的同时还需记录张量的换入换出时间点 $\{EvictTime, PrefetchTime\}$, 插入 $swap_point$ 列表中。

(6) 由于张量的换出会影响其子节点张量的重计算路径和开销, 需根据逐出张量 t 的信息更新 $t_candidate$ 中张量的重计算开销, 其算法与算法 3.2 类似, 不多赘述。

(7) 根据当前决策产生的 TSI_k 对当前 $t_candidate$ 中的张量进行重新排序, 更新序列 t_TSI 。

(8) 重复(2)到(7)的操作, 直到进行内存交换减少的内存空间占用大于目标内存需求量 $memory_target$ (正常模型训练占用的内存与内存阈值的差), 结束内存交换张量选择算法。

需要说明的是, 算法 3.1 中 22 行使用函数 $SortTSI(t_candidate, TSI_k)$ 对 TSI 从大到小排序, 具体来说, 首先根据公式(3-2)和(3-3)计算出 $t_candidate$ 中所有张量的 $TSI_{k+1,i}$ (其中 $k+1$ 代表第 $k+1$ 次决策, i 代表张量 i), 再对 TSI 从大到小排序, 得到的 t_TSI 为张量序列, 序列中按照 TSI 从大到小的排序存放对应张量的序号, 例如 t_TSI_0 为 TSI 最大的张量序号。

b) 混合方法选择阶段

混合方法选择阶段的混合张量选择算法如算法 3.2 所示, 其主要功能为: 对张量

的 VFM 进行排序, 得到 VFM 序列 t_VFM , 根据 VFM 值大小选择张量进行内存交换或重计算, 内存交换张量插入 t_swap 集合, 重计算张量插入 $t_recompute$ 集合。

算法 3.2: 混合选择算法

输入: 张量候选集合($t_candidate$), 目标内存需求量($memory_target$)

输出: 内存交换张量集合(t_swap), 重计算张量集合($t_recompute$)

```
1.  $t\_VFM \leftarrow InitVFM(t\_candidate);$ 
2. while  $memory\_target > 0$  do
3.    $t \leftarrow t\_VFM_i;$ 
4.    $t.extra\_count \leftarrow 1;$ 
5.   for  $rt$  in  $t\_recompute$  do
6.     if  $t \in rt.parent$  then
7.        $rt.parent \leftarrow t.parent;$ 
8.        $rt.RecomputeTime += t.RecomputeTime;$ 
9.        $t\_recompute.delete(rt);$ 
10.       $t\_candidate.add(rt);$ 
11.       $memory\_target \leftarrow memory\_target + t.size;$ 
12.    end
13.     $ChooseMethod(t);$ 
14.     $t\_candidate.delete(t);$ 
15.     $memory\_target \leftarrow memory\_target - t.size;$ 
16.    for  $cand$  in  $t\_candidate$  do
17.      if  $t \in cand.parent$  then
18.         $cand.parent \leftarrow t.parent;$ 
19.         $cand.RecomputeTime += t.RecomputeTime;$ 
20.      end
21.    end
22.     $t\_VFM \leftarrow sortVFM(t\_candidate);$ 
23. end
```

算法 3.2 的工作步骤如下：

- (1) 初始化 $t_candidate$ 集合中张量的 VFM，每个张量有两个 VFM，一个对应内存交换方法的 VFM，一个对应重计算方法的 VFM。将 VFM 从大到小排序，得到序列 t_VFM 。
- (2) 选择 t_VFM 最大的张量，根据该张量的数据依赖关系更新 $t_compute$ 集合中张量的计算路径，记录该张量所在的重计算路径上的重计算张量个数。
- (3) 若该张量对应的方案为内存交换，则插入 t_swap 中，若其对应的方案为重计算，则插入 $t_recompute$ 中，从 $t_candidate$ 中删除该张量。
- (4) 根据该张量的数据依赖关系更新 $t_candidate$ 中张量的计算路径和重计算时间 $RecomputeTime$ 。若 $t_candidate$ 中张量的子节点或父节点为重计算节点，还需更新其额外开销 $ExtraTime$ 。
- (5) 更新 $t_candidate$ 集合中张量的 VFM，对 VFM 重新排序，得到序列 t_VFM 。
- (6) 重复(2)到(5)的操作，直到被逐出张量减少的内存空间占用大于目标内存减少量，结束混合张量选择算法。

算法 3.2 中对于 VFM 的排序过程与 TSI 排序类似，利用的是公式(3-4)， t_VFM 中按照 VFM 从大到小的排序存放对应张量的序号。需要说明的是，内存交换方法的 VFM 计算公式与公式(3-4)是相同的，但其中的 $GenerationTime$ 为张量从 GPU 内存传输到主存的时间，即 $SwapTime$ ，所以内存交换的 VFM 为一个固定值，即 PCIe 的带宽的倒数 $1/PCIeBW$ 。

3.3 动态张量选择方法

3.2 节提出的精细粒度张量选择方法选择大小之和满足内存需求量(正常模型训练占用的内存与内存阈值的差)的张量进行逐出，但训练时内存空间占用的动态变化会影响决策的正确性，本节提出了动态张量选择方法解决该问题。

动态张量选择方法包含动态张量决策方法和多阶段张量选择方法。首先，内存交换方法在换入张量时需要额外分配内存空间，当选择进行内存交换的张量很大时，即使换出张量到主存中，仍然不能减少内存空间占用，而重计算方法无需为重新生成的

张量分配内存空间。3.3.1 节给出了动态张量决策方法，对 3.2 节的精细粒度张量选择方法进一步优化。其次，内存交换方法在换出张量时需要等待张量换出完成后才能释放内存空间，内存空间的延迟释放可能仍会造成内存空间不足的情况，3.3.2 节给出了多阶段张量选择方法，保证了决策结果的正确性。

3.3.1 动态张量决策方法

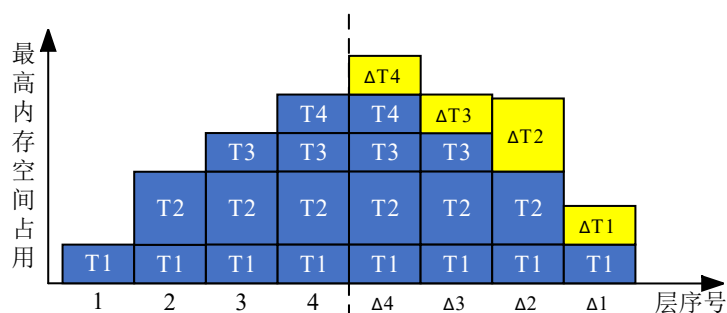


图 3-11 每层的最高内存空间占用

假设一个线性模型训练的前向传播中共生成四个张量，生成的时间顺序是 T1、T2、T3 和 T4，每一层的最高内存空间占用如图 3-11 所示，其中蓝色方块代表前向传播时产生的特征图张量占用的内存，黄色方块代表反向传播时产生的梯度图张量占用的内存，通常来说，同一层的梯度图张量与特征图张量的大小相同。整个训练过程中，内存空间占用的峰值是在反向传播第四层出现的。

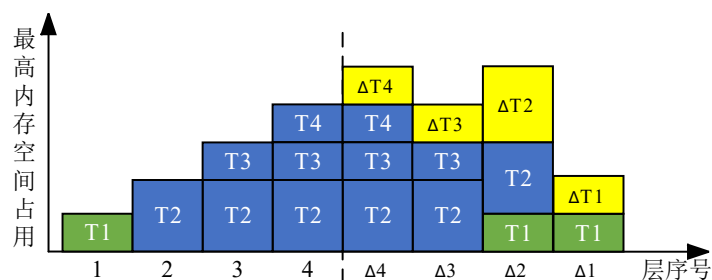


图 3-12 选择 T1 为内存交换张量

依据 3.2 节的张量选择方法，选择 TSI 最大的张量 T1 进行内存交换，训练时内存空间占用的动态变化如图 3-12 所示，其中绿色方块代表该张量被选为内存交换张量。由于将 T1 复制到了主存中，从前向传播的第二层开始，直到反向传播的第三层，T1 都不再占用 GPU 的内存空间(T1 需在被访问前提前换入 GPU 内存，因此在反向传播的第二层，会为 T1 分配内存空间，方便 GPU 内存与主存之间数据复制的进行)，

整个训练过程中的内存空间占用峰值明显下降，这说明内存交换方法对于减少 DNN 训练的内存空间需求有显著的成效。此时内存空间占用的峰值依旧出现在反向传播的第四层。

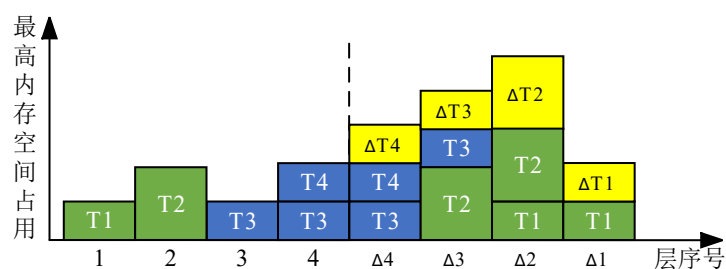


图 3-13 选择 T1 和 T2 为内存交换张量

接着，选择张量 T2 进行内存交换，训练时内存空间占用的动态变化如图 3-13 所示。将 T2 换出到主存中虽然可以进一步减少位于中部的部分层的最高内存空间占用，但对于整个训练过程来说，内存空间占用峰值并没有明显下降，且此时峰值出现在了反向传播的第二层。这是由于内存交换方法在换入张量时需要为内存交换张量分配内存空间，当该张量占用的内存空间较大时，会使得换出该张量所减少的内存空间占用变为无效的内存优化。发生此类情况时，即使继续选择其他张量进行内存交换或重计算，也无法改变内存空间占用的峰值。

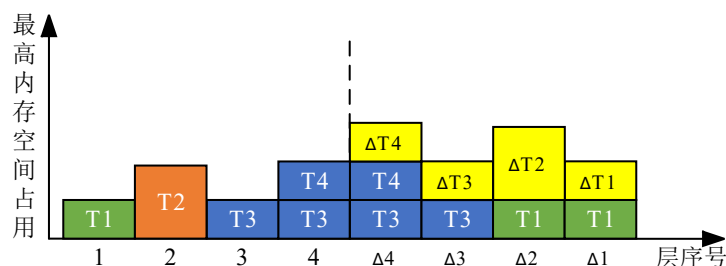


图 3-14 更改 T2 为重计算张量

观察发现，重计算方法在反向传播时将重新计算出的张量存放在寄存器中，并立即用于当前层的计算，因此不需要为重计算张量分配 GPU 内存空间。基于重计算方法的这个特点，当某个张量被选为内存交换张量时，内存空间占用峰值出现的位置发生了改变，则将该张量的决策更改为重计算。如图 3-14 所示，橙色方块代表该张量被选为重计算张量。当 T2 被选为重计算张量时，反向传播过程中不再需要为 T2 分配内存空间，此时内存空间占用的峰值又回到了反向传播的第四层，可以继续选择张

量进行内存交换和重计算。

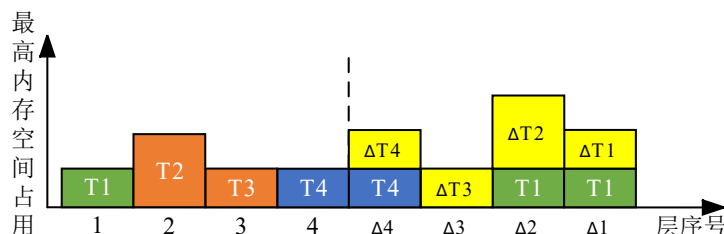


图 3-15 选择 T3 为重计算张量

值得注意的是，如果进一步选择张量 T3 进行重计算，会发现此时峰值再次出现在了反向传播的第二层，如图 3-15 所示。想要减少内存空间占用，需要将 T1 修改为重计算张量。这说明当内存空间占用峰值发生改变时，可以进一步将序号小于峰值层序号的张量修改为重计算张量，减少内存空间占用。

基于以上对于内存空间占用动态变化的观察，本工作提出动态张量决策方法。具体来说，当内存交换方法无法有效地减少内存空间占用时，找到内存空间占用峰值出现的层，对序号小于等于该层序号的张量进行重新决策，从而解决内存交换方法产生额外内存空间占用的问题。动态张量决策方法的算法如算法 3.3 所示，每进行一次决策后，计算内存空间占用减少量，若减少量少于被逐出的张量大小，则寻找内存空间占用峰值出现的层，将在该层计算时占用内存空间的张量的决策方案进行修改。

算法 3.3 的工作步骤如下：

(1) 进行一次决策，得到被选择的逐出张量 t 。记录决策前后的最大内存空间占用 MemoryFootprint_1 和 MemoryFootprint_2 。当两者的差值大于此次决策被逐出张量的大小，说明该决策的内存优化结果不如预期，需要修改决策，否则进行下一次决策。

(2) 若修改决策，首先将张量 t 插入 $t_candidate$ 序列中，并将张量 t 从 t_swap 或 $t_recompute$ 中删除，接着找到内存空间占用峰值出现的层，其输入张量序号记为 i ，如果张量 i 不是重计算张量，则修改为重计算张量。由于序号小于 i 的张量都可能会影响该层的内存空间占用，如果 i 是重计算张量，则进一步选择序号小于 i 的非重计算张量，并修改其决策结果。

(3) 若修改决策, 首先将张量 t 插入 $t_candidate$ 序列中, 并将张量 t 从 t_swap 或 $t_recompute$ 中删除, 接着找到内存空间占用峰值出现的层, 其输入张量序号记为 i , 如果张量 i 不是重计算张量, 则修改为重计算张量。如果 i 是重计算张量, 则进一步选择序号小于 i 的非重计算张量, 并修改其决策结果。

(4) 如果序号小于等于 i 的张量都是重计算张量, 说明该层的特征图张量已经大于显存容量, 无法通过内存交换或重计算方法减少内存空间占用。在这种情况下, 只能减小批处理大小或使用多个 GPU 进行训练。

算法 3.3: 动态张量决策算法

输入: 目标内存需求量($memory_target$)

输出: 内存交换张量集合(t_swap), 重计算张量集合($t_recompute$)

```
1.  while  $memory\_target > 0$  do
2.       $MemoryFootprint\_1 \leftarrow checkMemory();$ 
3.       $t \leftarrow t\_decision();$ 
4.       $memory\_target \leftarrow memory\_target - t.size;$ 
5.       $MemoryFootprint\_2 \leftarrow checkMemory();$ 
6.      if  $MemoryFootprint\_1 - MemoryFootprint\_2 < t.size$  then
7.           $t\_candidate.add(t);$ 
8.           $i \leftarrow findmaxLayer();$ 
9.          while  $i > 0$  do
10.             if  $i \notin t\_recompute$  then
11.                  $t\_recompute.add(i);$ 
12.                 break;
13.             end
14.              $i --;$ 
15.          end
16.      end
17. end
```

3.3.2 多阶段张量选择方法

内存交换方法在换出张量时并不会立即释放张量占用的内存空间,而是等待张量换出完成后才会释放张量占用的内存空间。该内存空间延迟释放的问题可能影响决策的正确性。

假设有一个五层的 DNN 网络,每层输入张量的大小如表 3-1 所示,进行累加后可以得到每层计算时的最高内存空间占用。假设内存阈值是 16,则整体训练的内存需求量为 8(用第五层的内存空间占用减去内存阈值),而张量 3 的大小为 8,逐出张量 3 看似是可以解决问题的(基于整体内存需求量进行的决策被称为整体选择方法),但在真实训练中,在前向传播阶段,内存交换张量会等待张量换出完成后才释放其占用的内存空间,因此,如果逐出张量 3,那么第 3 层计算过程中,最高内存空间占用仍然是 17(见图中下划线标注的数据),超过了内存阈值,会产生内存不足的问题。

表 3-1 整体选择方法的内存空间占用量

层序号	1	2	3	4	5
输入张量大小	2	7	8	3	4
最高内存空间占用(不逐出)	2	9	17	20	24
最高内存空间占用(逐出张量 3)	2	9	<u>17</u>	12	16

为了解决上述问题,本工作提出了多阶段选择方法。该方法将训练过程分为多个阶段,并针对每个阶段内存不足的情况选择内存交换张量。具体来说,首先找到第一个内存空间占用超过内存阈值的层(记为超量层),在序号小于超量层的张量中依据精细粒度张量选择方法和动态张量决策方法选择张量进行逐出,同时更新每层的最高内存空间占用。接着找到下一个超量层,重复以上操作,直到每层的最高内存空间占用都小于内存阈值时,此时的决策方案即为最终的决策方案。

对于表 3-1 中的例子,多阶段选择方法的工作流程如表 3-2 所示,在没有选择张量逐出时,第 3 层的最高内存空间占用为 17,超过了内存阈值 16,该层的内存需求量为 1,需要在张量 1 和张量 2 中选择合适的张量逐出,根据选择较小张量的原则,选择了张量 1 进行内存交换,得到了更新后的每层最高内存空间占用,第四层仍然

是超量层(最高内存空间占用为 18)，内存需求量为 2，此时选择张量 2 进行内存交换。进行了以上两个选择后，任何一层的最高内存空间占用都低于内存阈值，于是结束进一步的张量选择操作，得到的最终的决策方案，即逐出张量 1 和张量 2。

表 3-2 多阶段选择方法的内存空间占用量

层序号	1	2	3	4	5
输入张量大小	2	7	8	3	4
最高内存空间占用(不逐出)	2	9	<u>17</u>	20	24
最高内存空间占用(逐出张量 1)	2	7	15	<u>18</u>	22
最高内存空间占用(逐出张量 1 和 2)	2	7	8	11	15

将以上问题一般化，假设有 N 个张量($t_1, t_2, \dots, t_{N-1}, t_N$)，所有张量之和为 S ，根据整体选择方法，逐出其中 k 个张量($t_i, t_{i+1}, \dots, t_{i+k-1}$)， S_i 为第 i 层的最高内存空间占用，逐出张量大小之和为 ΔS ，内存阈值为 `memory_th`。推导公式如(3-5)所示，在减去 k 个张量的内存空间占用后，内存空间占用小于 `memory_th`，而 S_i 大于 `memory_th` 说明使用整体选择方法仍然会导致 OOM，因此，当 S_i 大于 `memory_th` 时需使用多阶段选择方法，即当第 i 个的张量为逐出张量，且张量大小大于第 $i+1$ 个到第 N 个张量中未逐出的张量之和时，第 i 层超量层，需使用多阶段选择方法，否则使用整体选择方法即可。

$$\begin{aligned}
 S &= t_1 + t_2 + \dots + t_N \\
 \Delta S &= t_i + t_{i+1} + \dots + t_{i+k-1} \\
 S - \Delta S &= t_1 + t_2 + \dots + t_{i-1} + t_{i+k} + \dots + t_N < \text{memory_th} \\
 S_i &= t_1 + t_2 + \dots + t_i \tag{3-5} \\
 \text{当} \quad S_i &> \text{memory_th} > S - \Delta S \\
 \text{则} \quad t_1 + t_2 + \dots + t_i &> t_1 + t_2 + \dots + t_{i-1} + t_{i+k} + \dots + t_N \\
 \text{即} \quad t_i &> t_{i+k} + \dots + t_N
 \end{aligned}$$

多阶段选择方法的算法如算法 3.4 所示。首先根据内存需求量使用整体选择方法进行选择与决策，接着，依据该决策结果找到超量层，对超量层之前的张量重新进行

选择和决策，完成后寻找新的超量层进行新一轮选择和决策，如此循环往复直到不再出现超量层， t_choose 即为最终逐出張量合集。

算法 3.4：多阶段选择算法

输入：DNN 数据流图(net)

输出：逐出張量集合(t_choose)

```
1.  $point \leftarrow -1, flag \leftarrow -2;$ 
2. while  $flag \neq point$  do
3.      $flag \leftarrow point;$ 
4.      $t\_choose \leftarrow decision(net, point)$ 
5.      $temp\_size \leftarrow 0;$ 
6.     for  $i \leftarrow 0$  to  $net.length() - 1$  do
7.         if  $net[i] \in t\_choose$  then
8.             for  $j \leftarrow i$  to  $net.length() - 1$  do
9.                 if  $j \notin t\_choose$  then
10.                     $temp\_size += net[j].size;$ 
11.                end
12.            end
13.            if  $net[i].size > temp\_size$  then
14.                 $point \leftarrow i;$ 
15.                break;
16.            end
17.        end
18.    end
19. end
```

算法 3.4 的工作步骤如下：

(1) $point$ 代表超量层序号，初始化为-1。 $flag$ 用于记录上一次决策时超量层的序号。当 $point$ 为-1 时，使用整体选择方法，得到逐出張量集合 t_choose 。

(2) 从第一个被逐出的张量开始计算, 逐个与其后面未被逐出的张量大小之和进行比较, 找到最先出现的张量, 记录其序号 `point`, 并根据 DNN 数据流图和 `point` 更新逐出张量集合。

(3) 重复步骤(2)的操作, 直到 `point` 与上一次决策的超量层序号 `flag` 相同。这表示不再出现新的超量层, 当前的逐出张量集合即为最优解。

需要说明的是, 本工作基于模拟训练的信息对内存空间占用的动态变化进行预估, 决策依旧是在训练前完成的。此外, 真实训练与模拟训练的时间和内存空间占用可能有些许出入, 但逐出的张量大小往往会大于内存需求量, 因此在训练过程中因为模拟误差导致内存不足的概率很小。

3.4 张量放置策略

内存交换方法打乱了张量的分配和释放的顺序, 导致部分被释放的内存空间无法被其他张量分配使用, 产生了内存碎片问题, 本节提出了张量放置策略解决该问题。

在训练 DNN 模型时, 需要在迭代过程中频繁地为张量分配和释放内存, 频繁调用 CUDA API 提供的 `cudaMalloc` 和 `cudaFree` 会产生大量不必要的开销。为解决这个问题, 现有工作常常使用 GPU 内存池技术^[55-57]。该技术会预先分配一个连续的内存块作为共享内存池, 并绕过操作系统进行相应的内存管理。预先分配的内存池会返回一个已分配但为空的地址列表。内存池通过两个列表管理内存的分配与释放, 空闲列表装载空闲内存块的地址, 已分配列表装载已分配内存块的地址。对于一个内存分配请求, 内存池从空闲列表中找到第一个足够大的空闲内存块, 返回其内存地址, 并更新空闲列表和已分配列表。对于一个内存释放请求, 内存池会在已分配列表中删除对应内存块, 并将该节点的地址插入空列表中。由于内存池为张量分配的是连续的地址空间, 当内存释放的顺序与内存分配的顺序不同时, 会产生内存碎片问题。

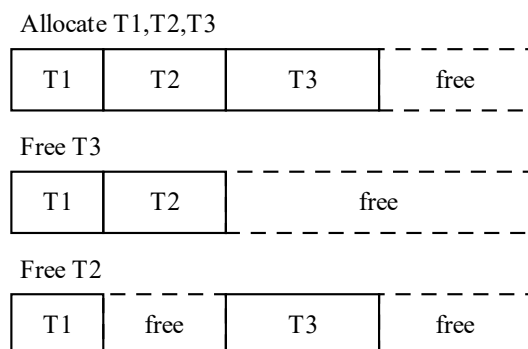


图 3-16 内存碎片

内存池提供的分裂和融合方法可以解决内部碎片，但无法解决内存交换等方法造成的外部碎片问题。如图 3-16 所示，按照 T1，T2，T3 访问顺序分配内存，如果不使用内存交换方法，在反向传播过程中，后访问的张量会先被释放，即 T3 先释放，由于为 T3 分配的内存空间和空闲的内存空间是相邻的，这两块内存空间会合并成为一个更大的页面，并不会产生内存碎片。若选择 T2 为内存交换张量，在前向传播期间，T2 会提前释放其内存空间，这部分内存空间无法与其他空闲内存空间融合，会导致内存碎片问题。当后续训练过程中张量所需的内存空间均大于这块内存空间时，内存池无法将该内存空间的地址分配给任何张量，因此这块内存空间的释放属于无效释放，并没有被其他张量所利用。

需要注意的是，只有数据传输与计算不同步时会产生碎片问题，以图 3-16 的内存分配释放过程为例，如果在每层计算完成时同步数据传输与计算，在前向传播的过程中，会等待 T2 完成换出后再为 T3 分配内存，此时 T2 占用的内存空间已经被释放，此部分内存空间会与相邻的空闲内存块融合，可以被其他张量(如 T3)利用。

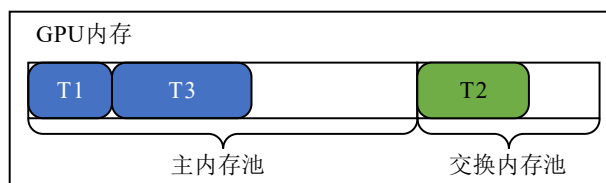


图 3-17 张量放置策略

为了解决内存碎片问题，本工作提出了张量放置策略。张量放置策略在训练开始前会通过 `cnmemInit()` 函数预先分配两个连续的内存池，如图 3-17 所示，分别为主内

存池和交换内存池。经过 3.2 节中的决策，模型在一次迭代过程中生成的张量分为三种，包括存储张量、内存交换张量和重计算张量。具体而言，存储张量是存储在内存中，不进行内存交换或重计算的张量，内存交换和重计算张量则为进行内存交换和重计算的张量。在前向传播过程中，将存储张量放置在主内存池中，将重计算张量和内存交换张量放置在交换内存池中；在反向传播过程中，将换入的内存交换张量放置在交换内存池。两个内存池的大小根据训练前的模拟训练所得到的信息进行分配。

对于内存交换张量，如果将其放在主内存池，待其换出后，后续没有合适大小的张量可以放置在此块内存空间则会造成此块内存空间空闲但不可用。如果将其放在交换内存池，由于内存交换张量频繁换入和换出，此块内存空间会被频繁分配和释放，通过内存池提供的内存融合方法，空闲块的大小也会随之改变，后续张量利用到空闲块的概率大大增加。对于重计算张量，在前向传播过程中，假设某一层的输入为重计算张量，在计算过程中，输入张量不可被释放，输出张量需要提前分配空间，即使在该层完成计算后立刻释放重计算张量的内存空间，也会造成内存碎片问题，因此选择将重计算张量放置在交换内存池中。在反向传播过程中，重计算生成的张量存放在寄存器中，无需占用内存空间。

3.5 精细粒度的动态内存管理方法实现

根据决策得到的 t_swap 集合、 $t_compute$ 集合、 $t_candidate$ 集合和 $swap_point$ 序列，为每个张量打上相应标签，得到新的模型训练数据流图。共有四种决策结果：NONE、TO_OFFLOAD_1、TO_OFFLOAD_2 和 TO_RECOMPUTE。其中 NONE 表示存储在 GPU 内存中，TO_OFFLOAD_1 表示进行内存交换且有对应的换出时间和换入时间，需根据换入换出时间进行内存交换，TO_OFFLOAD_2 表示按需进行内存交换，前向传播时在对应层换出，反向传播时在对应层上一层换入。TO_RECOMPUTE 表示进行重计算，反向传播时在被访问时进行重计算。

FDNN 采用了两个独立的 CUDA 流， $Stream_memory$ 和 $Stream_compute$ ，将深度神经网络训练过程中的计算操作与内存分配，释放和内存交换操作重叠起来。 $Stream_memory$ 管理着内存分配释放和内存交换两个功能， $Stream_compute$ 管理着

张量的计算和重计算两个功能。下面分四点具体阐述内存分配与释放、张量换出、张量换入和重计算功能的实现。

(1) 内存分配与释放

由于 CUDA 库提供的 `cudaMalloc()` 或 `cudaFree()` 只支持同步内存分配与释放，任意对于这两种命令的调用都会导致额外的同步开销。为了不被这部分开销影响实验结果，FDNN 采用了 Nvidia CNMeM^[58] 提供的开源异步内存分配与释放操作 `cnmemAlloc()` 和 `cnmemFree()`。在进行预估决策后，对优化后的最大内存空间占用进行估算，根据估算值分配两个内存池。当首次调用 `cnmemAlloc()` 时，会通过分裂的方法将内存池分为两个块，其中一个块适应分配所需的大小，为所分配的内存块，另一个块则为内存池剩余的部分。内存池使用双向链表将内存块的元数据保存下来，再次分配内存时，通过双向链表寻找空闲且合适大小的内存块，依然通过分裂的方式进行内存分配。当调用 `cnmemFree()` 时，需要查询该内存块前后是否有空闲块，如果有，则在释放该内存块的空间后，将双向链表中这几个空闲块进行合并，获得更大的空闲块以方便后续的内存分配请求。

(2) 张量换出

前向传播过程中，在某个张量换出前，首先需要调用 `cudaMallocHost()` 在主存中分配一块固定内存空间，这部分内存空间只能用于与 GPU 内存之间的数据传输，不能用于主机端的应用。接下来，`Stream_memory` 调用 `cudaMemoryAsync()` 通过 PCIe 总线将该张量无阻塞地传输到主存的固定内存中，该操作可以使数据传输与计算操作并行完成。但由于计算时间小于数据传输时间，对于在混合决策阶段选择的内存交换张量，则会在其对应的层开始计算时进行数据传输，由于作为输入的张量都是只读的数据，该张量的换出操作与其对应层的计算过程进行重叠不会影响实现的正确性。

此外，由于 FDNN 采取的是计算与数据传输不同步的方式，需要注册一个事件 `event_offload_done[i]`，该事件表示张量 i 换出已完成。在第 i 层计算完成后(通过调用 `cudaStreamSynchronize(Stream_memory)` 确保计算完成)，将启动一个单独的线程，用于在 `cudaEvent(event_offload_done[i])` 发生后释放张量 i 占用的 GPU 内存。为了跟踪张量是否被释放，使用了一个信号量(`sem_sync_offload`)，每启动一个单独的线程会递

增该信号量。该线程被分离，以便线程资源在它完成后立即释放，而无需调用 `pthread_join()`。具体流程如下：

(1) 张量 i 在某层进行换出，在第 i 层结束时启动一个线程等待 `cudaEvent(event_offload_done[i])` 发生，递增信号量 `sem_sync_offload`。

(2) `cudaEvent(event_offload_done[i])` 发生后可以释放对张量 i 占用的 GPU 内存，释放完成后递减信号量。

(3) 在所有层的前向传播之后，主线程需要确认 `sem_sync_offload` 为 0 后才能进入反向传播，保证了反向传播开始之前内存交换张量都从 GPU 内存中卸载和释放。

此外，由于 FDNN 采用分阶段选择方法，需要在超量层前完成部分张量的卸载和释放，因此只有等到 `sem_sync_offload` 为 0 才能开始超量层的计算。

(3) 张量换入

与张量换出类似的，`Stream_memory` 调用 `cudaMemoryAsync()` 通过 PCIe 总线将换出的张量无阻塞地传输到 GPU 内存中，并使数据传输与反向传播的计算并行完成。由于深度神经网络的训练过程中对于张量的访问呈现对称的模式，因此在反向传播过程中要以与张量换出相反的顺序进行张量换入操作。同时，张量的换入需要在对应层开始计算前就完成换入，因为前向传播产生的张量要作为反向传播的输入。

进一步地，为了在计算与数据传输不同步的情况下，保证训练的正确性，注册一个事件 `event_prefetch_done[i]`，该事件表示张量 i 换入已完成。反向传播的第 i 层需要等待该事件发生后，才能开始计算。

(4) 重计算

在前向传播过程中，第 i 层完成计算后，便立即释放重计算张量占用的 GPU 内存。在进行预估决策时，会记录重计算张量的父节点张量，在反向传播过程中，调用前向传播过程中所用到的计算函数，根据父节点张量对张量进行重新计算。在重计算张量时需要调用 `cudaStreamSynchronize(Stream_memory)` 确保重计算的完成，保证其结果已被正确计算出，可以用于后续层的计算。

3.6 本章小结

本章阐述了基于张量的精细粒度动态内存管理方法 FDNN 的设计与实现，给出了方法的整体架构和每个模块的主要算法和原理。对于该方法的论述主要分为三个部分：第一，基于对内存交换和重计算方法的分析，详细阐述了内存交换和重计算的权衡策略的工作原理，展示了精细粒度张量选择方法整体工作流程；第二，基于对内存空间占用动态变化的分析，详细阐述了动态张量决策方法和多阶段张量选择方法的工作原理；第三，基于对内存交换方法导致的内存碎片问题的分析，阐述了张量放置策略的工作原理。最后阐述了 FDNN 实现的原理和方法。

4 性能测试与结果分析

本章对于精细粒度的动态内存管理方法 FDNN 进行了性能测试。介绍了 FDNN 的测试环境、对比方案和测试用例。测试了 FDNN 相较于只使用了内存交换方法或重计算方法的 vDNN 和 InPlace-ABN(IPABN)的性能提升。分析了动态张量选择方法对张量决策和内存空间占用变化的影响。测试了 FDNN 相较于粗粒度内存管理方法 SuperNeurons 的性能提升。

4.1 测试环境

本实验在 Ubuntu 操作系统上使用 C++语言和 g++编译器完成各功能模块的开发。Ubuntu 版本为 18.04 LTS, g++版本为 7.5.0。实验平台配备了频率为 2.10GHz, 内存为 187 GB 的 Intel(R) Xeon(R) Gold 6230R 型号的处理器的处理器, 内存为 16 GB 的 NVIDIA Tesla V100 GPU 和 PCIe3.0×16。CUDA 的工具安装包 CUDA Toolkit 的版本为 10.2, 深度学习计算软件库 cuDNN^[59]的版本为 7。FDNN 使用 Nvidia CNMeM^[58]提供的开源异步内存分配与释放库, 根据对于 Pytorch 采用的内存分配与释放库的对比, 两者的内存管理方法基本一致, 为了方便实验观察, FDNN 选择了编程相对简单的 CNMeM 工具, 理论上本文提到的方法可以移植到 Pytorch 框架中。

4.2 对比方案与测试用例

(1) 对比方案

实验对比的工作共有三个: vDNN、IPABN 和 SuperNeurons。vDNN^[23]只使用了内存交换技术, 选择计算时间长的卷积层的输入作为内存交换的对象, 能显著减少内存开销, 但 vDNN 在每层计算结束时对计算和数据传输的同步造成了较高的时间开销。IPABN^[52]只使用了重计算技术, 该工作将批处理归一化层和激活层的计算合并, 不保留中间的计算结果, 在反向传播时通过逆计算重新计算出来, 也能显著减少内存开销, 但 IPABN 只支持可逆的激活层的重计算, 限制了 IPABN 的应用范围。SuperNeurons^[24]使用了内存交换和重计算两项技术, 该工作选择卷积层输入进行内存

交换的同时，选择计算时间短且内存空间占用量大的激活层、池化层和批处理归一化层的输入进行重计算，实现比 vDNN 更小的内存空间占用的同时，减少时间开销。

实验与前两个工作对比的目的是证明 FDNN 的性能收益并不来源于某一种方法，而是基于张量特点对于两种方法权衡选择的结果；与 SuperNeurons 对比的目的是测试 FDNN 相较于粗粒度内存管理方法的性能提升。

实验使用两个指标进行性能测试，包括最大批处理大小和训练速度。GPU 内存管理方法的首要目标是减少一定的内存空间占用，用于测试内存空间占用减少量的指标是训练能达到的最大批处理大小，最大批处理大小越大，说明 FDNN 减少的内存空间占用越多。在提高内存使用率的同时，也要避免高额的时间开销对训练造成影响，因此用于测试速度开销的指标是计算的吞吐量，即每秒训练的样本数量，每秒训练的样本数量越多，训练的速度越快。

(2) 测试用例

训练深度神经网络需要大量的数据集，实验中采用标准的数据集 Mnist^[60]。Mnist 数据集中包含 60 000 个训练图像和 10 000 个测试图像，主要用于识别手写数字，该数据集的数据量大小适中，训练的目标明确，任务简单，不会因为训练过程过于复杂而影响对于内存空间占用的观察。

表 4-1 测试模型

模型名称	内存空间占用(256)	层数	线性或非线性
AlexNet	0.98 GB	12	线性
VGG16	20.05 GB	16	线性
VGG19	20.05 GB	19	线性
ResNet50	10.68 GB	50	非线性

本实验选择进行测试的模型为深度神经网络应用中常用的模型 AlexNet, VGG16, VGG19 和 ResNet50。这些模型的配置(层数、层类型和参数等)与 vDNN 中使用的模型一致，表 4-1 展示了模型的结构和在批处理大小为 256 的内存空间占用情况。AlexNet 层数较少，训练过程中对于内存空间占用较小，可以在 16 GB 的 GPU 中轻松地进行大批量训练，此模型用于观察 FDNN 中每种优化方法的决策结果以及性能

提升的原因。VGG16 和 VGG19 层数较少，训练过程中的张量的内存空间占用量大，批处理大小为 256 时，其最大内存空间占用达到了 20 GB，这两个模型用于测试 GPU 内存不足时，FDNN 产生的性能收益。这三个模型均为线性模型，而 ResNet50 为非线性网络，层数较多，结构较复杂，包含 49 个卷积层和一个全连接层，该模型代表有一定深度和广度的深度神经网络，在训练过程中张量的访问模式更为复杂，用于测试 FDNN 的扩展性。

4.3 张量选择结果及性能分析

4.3.1 与内存交换方法的性能对比

由于 vDNN 在每层计算结束时对计算和数据传输的同步造成了较高的时间开销，本节对比了不进行同步时 vDNN 的与 FDNN 的时间开销对比，证明 FDNN 相较于 vDNN 的性能收益并不完全来源于取消同步操作。此外，实验将内存阈值设置为 vDNN 训练模型所需的内存空间大小，FDNN 根据该内存阈值做出决策，目的是避免不同的内存空间占用对于性能测试的影响。

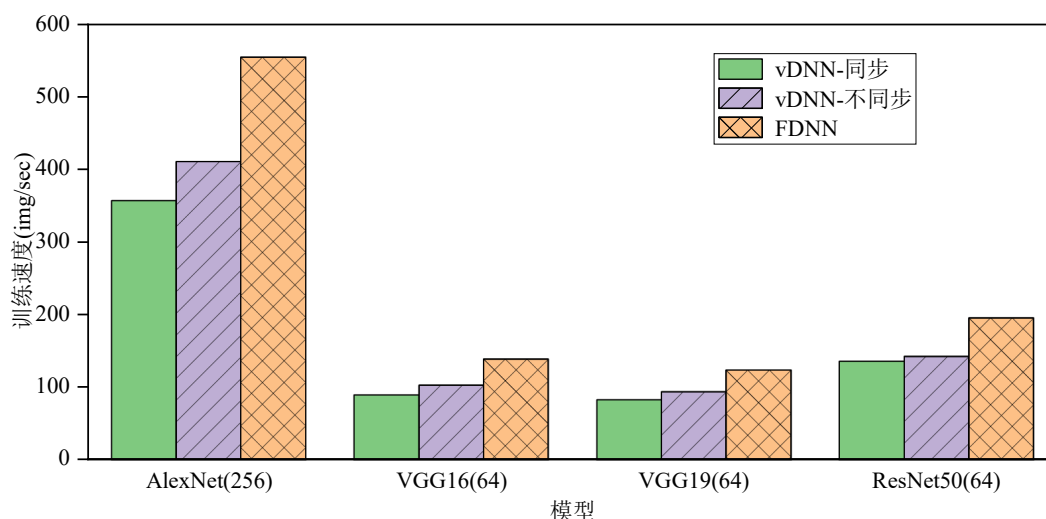


图 4-1 与 vDNN 的训练速度对比

与 vDNN 工作的对比测试结果如图 4-1 所示，测试了四种 DNN 模型是否进行同步的训练速度，以及 FDNN 获得的训练速度提升。横坐标模型名称后括号中的内容为批处理大小。四种 DNN 模型不同步方案相较于 vDNN 同步方案的训练速度提

升了 10.89%, 13.02%, 11.7%和 4.6%。其中, ResNet50 不同步方案带来的收益较小的原因是 ResNet 中卷积层占比大, 而卷积层的计算时间较长, 计算时间与传输时间的重叠比例高。FDNN 决策方案相较于 vDNN 同步方案的训练速度提升了 40.2%, 35.6%, 33.1%和 30.6%, 说明选择部分计算开销较小的张量进行重计算, 可以解决选择过多内存交换张量导致的传输时间过长的问题。

4.3.2 与重计算方法的性能对比

由于 FDNN 进行决策时, 张量的重计算开销会随着决策的进行发生改变, 本小节将不考虑重计算开销变化的决策方法记为 FDNN_CTD (Compute-Time based Decision), 将考虑重计算开销变化的决策方法记为 FDNN_RTD (Recompute-Time based Decision), 对比 FDNN_RTD 相较于 FDNN_CTD 的性能提升。此外, 实验将内存阈值设置为 IPABN 训练模型所需的内存空间大小, FDNN 根据该内存阈值做出决策, 目的是避免不同的内存空间占用对于性能测试的影响。

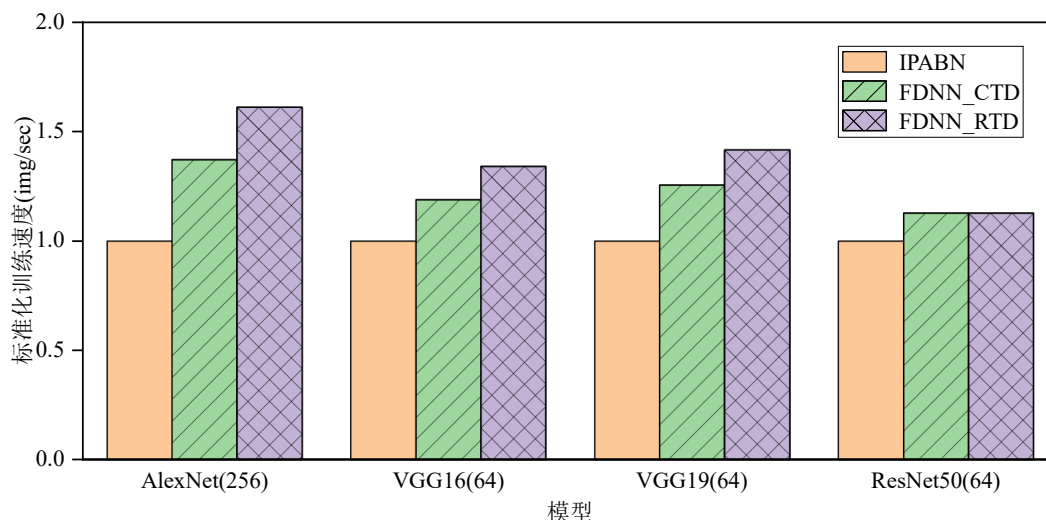


图 4-2 与 IPABN 的标准化训练速度对比

与 IPABN 工作的对比测试结果如图 4-2 所示, 测试了四种 DNN 模型使用 FDNN_CTD 方案和 FDNN_RTD 方案进行决策的标准化训练速度(与 IPABN 方案的比值)。FDNN_CTD 方案相较于 IPABN 方案的训练速度提升了 37.02%, 18.75%, 25.47%和 12.7%, FDNN_RTD 方案相较于 IPABN 方案的训练速度提升了 61%, 33.92%, 41.5%和 12.7%。FDNN_RTD 方案相较于 FDNN_CTD 方案在训练速度上有进一步的

提升,其中,ResNet50 的 FDNN_RTD 方案和 FDNN_CTD 方案的训练速度相同,是因为依据 FDNN_CTD 方案选择的重计算张量的父节点没有被逐出内存,因此重计算开销保持不变,FDNN_RTD 的决策方案也与 FDNN_CTD 一致。以上性能提升说明选择部分张量进行内存交换可以有效减少重复计算带来的额外时间开销。

4.3.3 动态张量决策结果与分析

实验对比了只使用精细粒度张量选择方法(记为 FDNN_s)和进一步使用了动态张量选择方法(记为 FDNN_d)的决策结果和内存空间占用变化过程。

实验设置批处理大小为 256,对 AlexNet 模型进行训练,训练中一次迭代过程的数据流图如图 4-3 所示,蓝色节点代表特征图张量,黄色节点代表梯度图张量,箭头代表数据依赖关系,AlexNet 共有 12 层,图中标号 1 到 12 的张量为前向传播过程中每层的输入张量,属于特征图张量;标号 $\Delta 12$ 到 $\Delta 1$ 的张量为反向传播过程中每层的输出张量,属于梯度图张量。

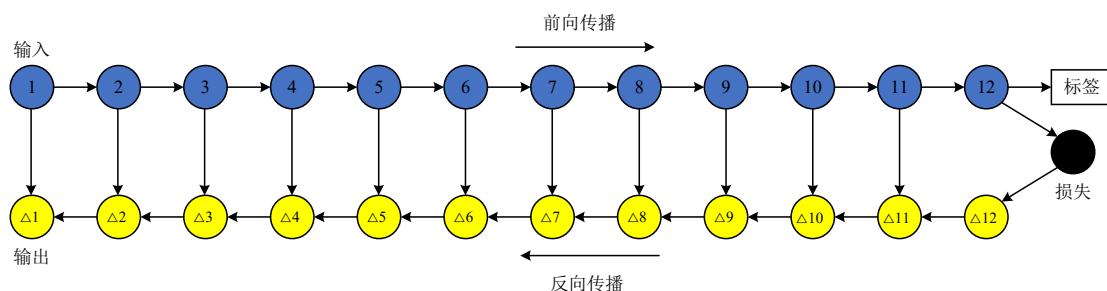


图 4-3 AlexNet 数据流图

AlexNet 在批处理大小为 256 时,保存所有张量所需的内存空间大小为 1 002 MB,而将张量全部进行重计算的内存需求空间为 503 MB(即完成训练的最小内存空间占用)。实验设置内存阈值为 503 MB,在此极限内存阈值下,FDNN 一定会使用动态张量选择方法进一步减少内存空间占用,这样的设置方便比较 FDNN_s 和 FDNN_d 方法的异同。

张量的决策共分为四种,即存储、重计算、内存交换 1 和内存交换 2。其中内存交换 1 是在内存交换选择阶段做出的选择,内存交换 2 则是在混合方法选择阶段做出的选择。由于决策是一个循环迭代的过程,每次迭代根据估算的性能开销选取或更新一个张量进行内存交换或重计算,可以形成一种决策方案。实验随机选取了

FDNN_s 决策过程中的一次决策结果与决策完成后的决策结果进行对比,如图 4-4 和图 4-5 所示。随着决策的进行,张量 3 和张量 5 始终为内存交换 1 张量,说明内存交换选择阶段进行的决策不会改变,而张量 7 被重新决策为内存交换张量,这是由于张量 7 的父节点张量被选为内存交换张量,重计算开销大大增加。FDNN_s 最终决策结果显示,为了避免大量计算时间开销,大部分张量都被决策为内存交换张量(第一层和最后一层的输入张量不参与决策,因为第一层的输入张量需作为重计算张量的计算源,而最后一层为 softmax 层,是一个简单的映射函数,一般会与倒数第二层一起执行)。

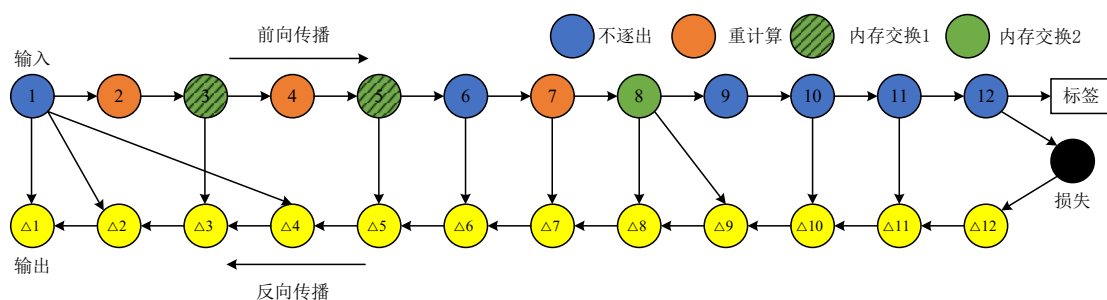


图 4-4 基于 FDNN_s 的决策中结果

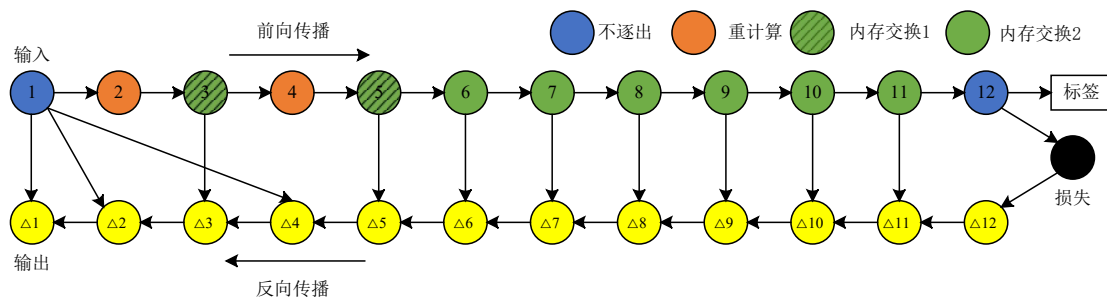


图 4-5 基于 FDNN_s 的决策后结果

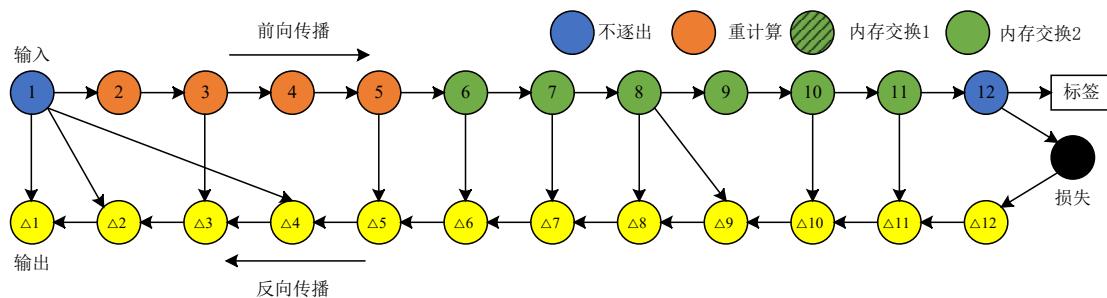


图 4-6 基于 FDNN_d 的决策后结果

由于内存交换张量在换入张量时需要分配额外内存空间,大量张量进行内存交换

必定会带来额外的存储开销，而重计算方法重新计算出张量时并不需要存放到 GPU 内存中，因此 FDNN_d 在决策的过程中对内存空间占用峰值进行了预估计算，将部分内存交换张量更新为重计算张量，如图 4-6 所示，张量 3 和 5 被重新决策为重计算张量。

为了说明 FDNN_d 有效减少内存空间占用的原因，实验根据以上三种决策结果进行训练，记录了训练中内存占用变化情况。如图 4-7 所示，AlexNet 共有 12 层，前向传播和反向传播共 24 个计算步骤，图中 1 到 12 步骤为前向传播过程，13 到 24 步骤为反向传播过程，图中蓝色曲线表示保存所有张量于 GPU 内存中，称为全存储方案。橙色和绿色曲线分别为基于 FDNN_s 的决策中和决策后方案的内存空间占用变化。紫色曲线为内存阈值为基于 FDNN_d 的决策后方案的内存空间占用变化。五角星标识的点为对应曲线决策结果的训练时内存空间占用峰值，内存空间占用峰值代表该决策训练所需的内存空间。

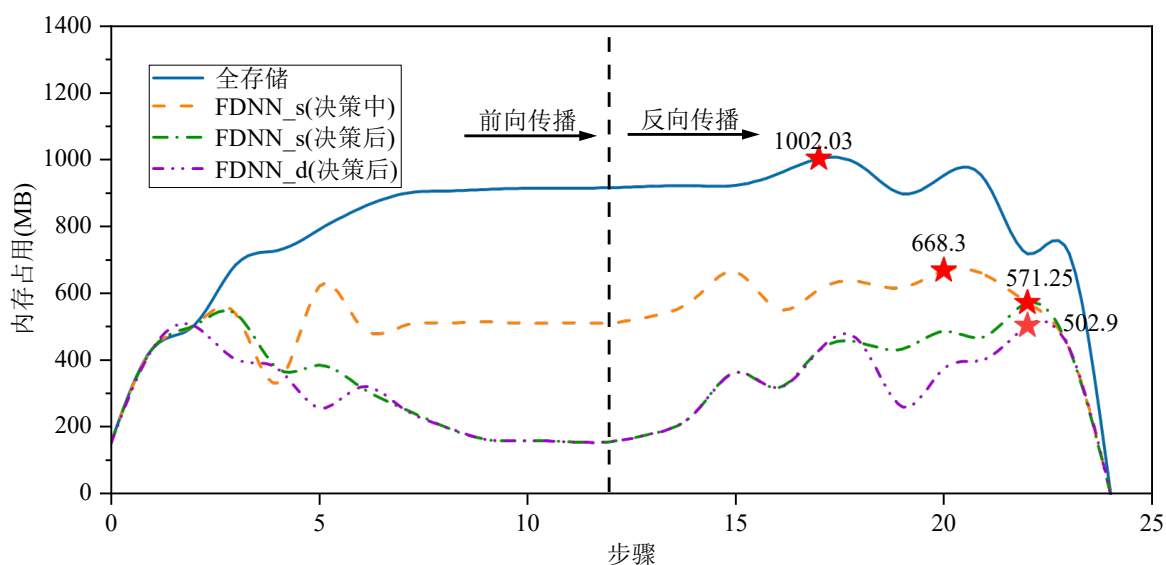


图 4-7 内存空间占用变化情况

基于对图 4-7 的观察，可以得出以下结论：

第一，DNN 训练的内存空间占用呈现相对对称的变化趋势。对于全存储方案，内存空间占用在前向传播过程中呈增加趋势，在反向传播过程中呈减少趋势，较高的内存空间占用集中在前向传播的后半部分和反向传播的前半部分。这是由于前向传播过程中不断计算生成特征图张量，而特征图张量作为反向传播对应层的输入，需要

等到该层完成梯度的计算才能释放。在前向传播开始与反向传播末尾的步骤中, FDNN_s 和 FDNN_d 决策方案的内存空间占用变化与全存储方案基本一致, 而中间步骤的内存空间占用显著减少, 这是由于 FDNN 选择逐出了部分训练过程中产生的特征图张量。此外, 反向传播中的内存空间占用大于前向传播过程中的内存空间占用, 这是由于反向传播过程中还产生了新的内存空间占用, 即梯度图。

第二, 相较于 FDNN_s, FDNN_d 进一步减少了内存空间占用, 而且保证了决策的正确性。首先, 对比 FDNN_s 的决策中和决策后方案的内存空间变化曲线。当逐出部分张量时, 中间步骤的内存空间占用减少, 有效减少了内存空间占用的峰值, 但继续选择张量逐出时, 内存空间占用峰值不再集中于中间步骤, 而是受内存交换方法换入张量所需的额外内存空间的影响, 逐出了所有张量后, 内存空间占用的峰值为 571.25 MB, 依旧大于内存阈值, FDNN_s 方案不再有效。接着, 对比 FDNN_s 和 FDNN_d 的决策后方案的内存空间变化曲线。FDNN_d 和 FDNN_s 在步骤 7 到 18 的内存空间占用基本一致, 这是由于两种决策方案对于张量 6 至 11 的决策是相同的。步骤 19 至 24 中, FDNN_d 和 FDNN_s 的内存空间差值即换入张量时额外分配的内存空间, FDNN_d 选择了张量 3 和 5 进行重计算, 无需为张量 3 和 5 分配额外的内存空间, 内存空间占用峰值下降至 502.9 MB, 小于内存阈值。此外, 步骤 3 至 6 中, FDNN_s 的内存空间占用大于 FDNN_d, 这是由于换出张量时需等待张量换出完成后才能释放内存空间。

4.3.4 与粗粒度内存管理方法的性能对比

(1) 内存空间占用减少量测试

图 4-8 展示了四种模型使用不同的内存管理方法所能达到的标准化最大批处理大小(与全存储方案的比值), 相比于全存储、vDNN、IPABN 和 SuperNeurons 方法, FDNN 平均能实现 3.39、2.13、1.73 和 1.59 倍的批处理大小的提升。SuperNeurons 和 FDNN 相较于只使用内存交换的 vDNN 工作和只使用重计算的 IPABN 工作, 能实现更高的批处理大小, 说明对于内存交换和重计算方法的权衡可以有效减少内存空间占用。相较于 SuperNeurons, FDNN 实现了更高的批处理大小, 说明相较于粗粒度内存管理方法, 细粒度的内存管理方法可以有效减少内存空间占用。此外, 由于不同的

模型层数和结构不同,FDNN 提高的批处理大小的幅度不同,以 VGG16 和 ResNet50 模型为例,ResNet50 层数深,能够逐出的张量也较多,因此性能提升幅度高,而 VGG16 层数较少,且部分张量占用的内存空间大,即使逐出所有张量,依旧需要提供较大的内存空间给每层计算需要用到的张量,因此性能提升有限。

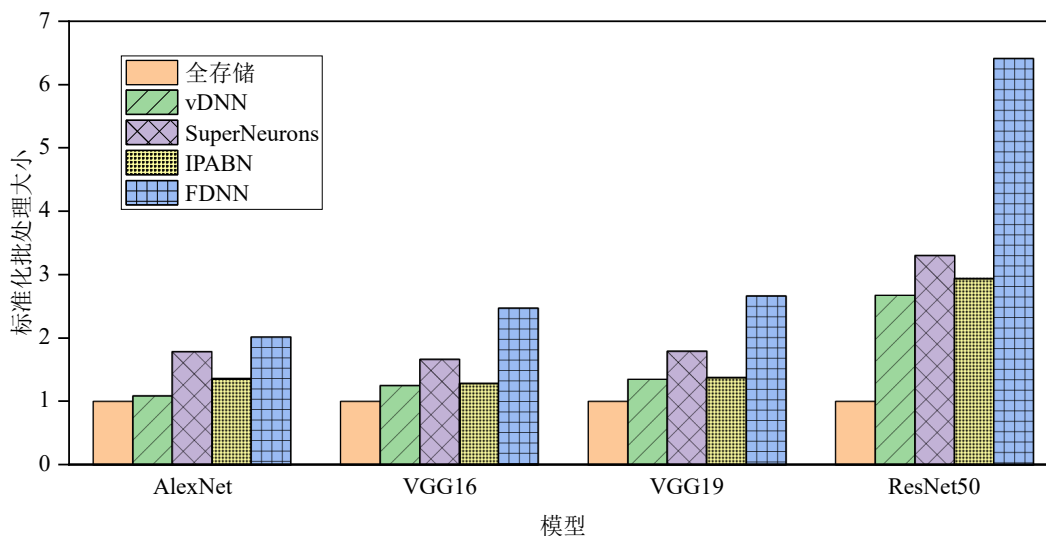


图 4-8 标准化最大批处理大小

(2) 训练速度测试

图 4-9 展示了四种模型在不同批处理大小下不同内存管理方法的训练速度。对于所有模型,进行任何批处理大小的训练,FDNN 总能实现最快的训练速度(由于不同方案支持的最大批处理大小不同,图中每条曲线的批处理大小范围也不同)。相较于 vDNN、IPABN 和 SuperNeurons 方案,FDNN 的训练速度最高能提升 264%、38%和 56%,最少能提升 39%、15%和 6%。vDNN 训练速度总是最慢,这是由于 vDNN 在每层计算结束都对数据传输与计算操作进行同步,造成了显著的同步开销,在层数较多的 ResNet50 中,同步开销更加明显。此外,由于 vDNN 和 IPABN 为静态决策方法,总是逐出相同的张量,这两种方法在不同批处理大小下的性能一直保持不变。而 SuperNeurons 会根据性能开销选择性地保存重计算路径中的部分张量,因此训练速度会发生变化。

观察 FDNN 的训练速度曲线,会发现当批处理大小增大到一定程度后,FDNN 的训练速度随着批处理大小的增大而减小,这是由于使用较大的批处理大小进行训练

时,内存需求量大,需要选择更多的张量进行重计算,最大限度地减少内存空间的占用,但重计算方法会引入大量的重复计算,从而降低了训练速度。使用较小的批处理大小进行训练,内存交换方法分配的额外内存空间对整体内存空间占用峰值影响不大,因此可以比较选择时间开销最小的方案,实现更快的训练速度。

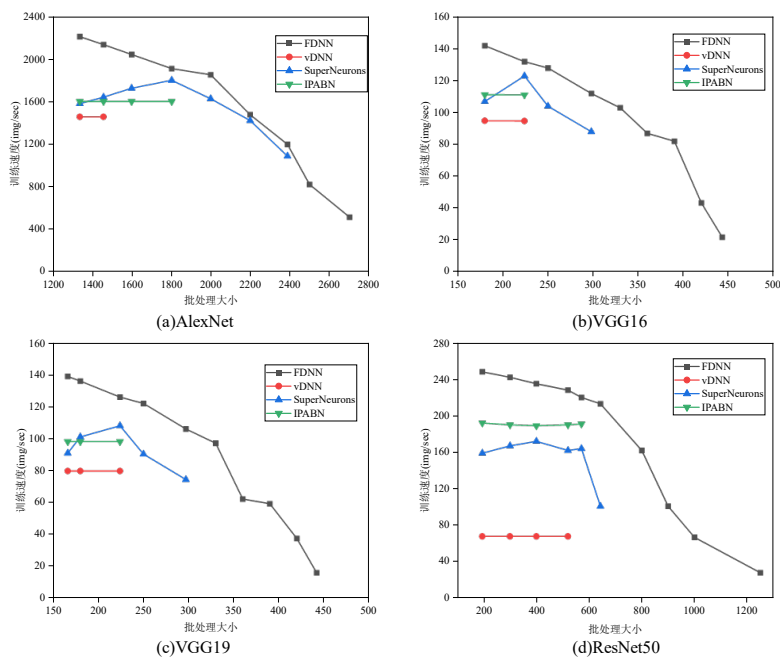


图 4-9 训练速度(不同方法支持的最大批处理大小不同,横坐标跨度不同)

4.4 本章小结

本章对面向深度神经网络的精细粒度动态内存管理方法 FDNN 进行了性能测试与分析。首先介绍了测试使用的实验环境与配置,并描述了测试使用的模型与对比方案。接着,测试了 vDNN、未采取同步操作的 vDNN 和 FDNN 决策方案的训练速度,实验结果证明 FDNN 可以在取消同步操作的基础上进一步提升训练速度。然后,测试了依据重计算路径变化更新重计算决策带来性能提升,实验结果证明了该更新方法可以得到进一步的性能提升。此外,测试分析了动态张量决策方法的决策结果及其训练内存空间占用变化,说明了动态张量选择方法的有效性和正确性。最后,从训练能达到的最大批处理大小和训练速度两方面,测试了 FDNN 相较于三个对比工作 vDNN、IPABN 和 SuperNeurons 在存储和性能两方面的收益,实验结果表明,相比

于这些工作，FDNN 支持最大的训练批处理大小和同一批处理大小下最快的训练速度。

5 总结与展望

5.1 论文总结

随着深度神经网络的深度和广度越来越高,神经网络训练的计算量和需要的内存空间也越来越大,这对训练设备的计算和存储性能提出了更高的要求。GPU 因其强大的计算能力成为训练深度神经网络模型的常用设备,但单个 GPU 设备内存容量有限,限制了对于复杂深度神经网络结构的探索和发展。现有工作常使用内存交换和重计算等技术,利用深度神经网络训练规律的重用模式,通过在前向传播中释放部分数据的内存空间,并在反向传播重新生成的方式,用暂时的内存需求代替持续累积的内存需求,使得在单个计算加速器中训练超过内存容量的模型成为可能。

由于内存交换方法和重计算方法分别利用存储资源和计算资源进行内存管理优化,可以同时使用两种方法以获得更高的内存利用率,但不同的决策方案会带来不同的存储和时间开销。现有的工作根据不同类型的层的特征对张量进行决策,但基于层的粗粒度决策方案没有量化比较两种方法的性能开销,无法根据内存需求的不同做出性能开销最小的决策。此外,训练过程中内存空间占用的动态变化还影响了决策结果的正确性。为了提供可量化的低开销决策方案,本文提出了精细粒度的动态内存管理方法 FDNN,主要包括以下工作及贡献:

(1) 分析量化了内存交换和重计算方法的性能开销。研究工作分析了不同张量在不同时间点进行内存交换的存储与时间开销,优先选择访问时间间隔长且内存空间占用小的张量进行内存交换。在不同张量决策方案中,根据张量的重计算路径与时间开销的变化,优先选择计算时间小且内存空间占用大的张量进行重计算,并在选择重计算张量的同时更新每个张量的重计算路径,保证重计算张量选择的正确性。

(2) 设计了精细粒度张量选择方法。研究工作首先利用主存与显存间的数据复制与张量计算并行执行的特点,优先选择内存交换张量以隐藏内存交换的时间开销。当内存交换开销无法被计算时间隐藏时,优先选择 VFM(重计算时间或换入张量时间与张量大小的比值)最小的张量进行内存交换或重计算。此选择方法可以根据训练的内存空间需求量生成时间开销最小的决策方案。

(3) 设计了动态张量选择方法。内存交换方法换入张量需要分配额外内存空间，而重计算生成的张量无需存放在 GPU 内存中，可以进一步减少内存空间占用。当使用内存交换方法无法进一步减少内存空间占用时，将造成内存空间占用峰值的内存交换张量更新为重计算张量。此外，内存交换方法在换出张量时有滞后性，没有及时换出张量可能导致内存不足，在决策时需要考虑张量的换出时间，进行多阶段决策。

(4) 设计了张量放置策略。内存交换方法打乱了张量分配与释放的顺序，造成了内存碎片问题。将内存交换张量与其他张量放置在不同的内存池中，使得内存交换张量与存储在 GPU 内存中的张量分配与释放互不干扰，减少内存碎片产生的概率。

(5) 使用多个模型测试了 FDNN 的性能开销。测试结果表明，相较于 vDNN、IPABN 和 SuperNeurons，FDNN 在支持训练更广更深的模型的同时保持了高训练速度。内存空间占用方面，FDNN 支持的最大训练批处理大小平均提升了 2.13、1.73 和 1.59 倍。在训练速度方面，FDNN 在相同批处理大小下，训练速度最高能提升 264%、38%和 56%，最少能提升 39%、15%和 6%。

5.2 展望

本文对于粗粒度内存管理方法的一些问题提出了相应的解决方案，但 FDNN 仍然存在一些不足和需要改进优化的地方。

(1) FDNN 的实现依赖于提前获取 DNN 模型的数据流图及其训练信息。在目前流行的深度学习框架中，可以基于声明式或命令式编程进行训练。声明式编程会在训练开始前构建模型的计算图，而实际的计算则是在需要时才会进行，在这种编程模式下，某些数据处理的优化方式可以很方便地应用于程序，FDNN 也是如此。然而，命令式编程会根据当前操作立即返回具体的值，并不需要构建数据流图，FDNN 在这种情况下无法获得 DNN 的训练信息，也就无法进行预估决策。一种解决办法是根据较小的批处理大小下的 DNN 训练信息对大批量 DNN 训练信息进行估计模拟，这种方法需要找到不同批处理大小间训练信息的联系，如果对于大批量 DNN 训练信息的估计准确，那么就可以对任意编程模式的 DNN 训练进行有效内存优化。

(2) 将 FDNN 应用于分布式深度神经网络训练。FDNN 提高了单一设备的内存利用率，但在多个设备并行训练的过程中，需要频繁地同步参数^{[61][62]}，而一次训练过程包含数百万次迭代，同步参数带来的时间开销是 FDNN 很难弥补的。RDMA^{[63][64]}常用于设备间的快速数据传输，如果能够将 RDMA 技术和 FDNN 有效地结合起来，那么可以在有限的 GPU 内存空间中，进一步实现超大模型的训练，同时保证快速的训练速度。

参考文献

- [1] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep Learning. *Nature*, 2015, 521(7553):436-444
- [2] Alex Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton. ImageNet Classification with Deep Convolutional Neural Networks. *Communications of the ACM*, 2017, 60(6): 84–90
- [3] 张顺, 龚怡宏, 王进军. 深度卷积神经网络的发展及其在计算机视觉领域的应用. *计算机学报*, 2019, 42(3): 453-482
- [4] Szegedy, Christian, Alexander Toshev, Dumitru Erhan. Deep Neural Networks for Object Detection. In: *Proceedings of Advances in Neural Information Processing Systems 26 (NIPS 2013)*. Lake Tahoe, Nevada, United States, December 5-10, 2013: 2553-2561
- [5] Sergey Ioffe, Christian Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. In: *Proceedings of the 32nd International Conference on Machine Learning (PMLR 2015)*. Lille, France, July 6-11, 2015: 448-456
- [6] Ronan Collobert, Jason Weston, Léon Bottou, Michael Karlen, Koray Kavukcuoglu, Pavel P. Kuksa. Natural Language Processing (Almost) from Scratch. *The Journal of Machine Learning Research*, 2011, 12: 2493-2537
- [7] 余凯, 贾磊, 陈雨强, 徐伟. 深度学习的昨天、今天和明天. *计算机研究与发展*, 2013, 50(9): 1799-1804
- [8] Alex Graves, Jürgen Schmidhuber. Framewise Phoneme Classification with Bidirectional LSTM and Other Neural Network Architectures. *Neural Networks*, 2005, 18(5-6): 602-61
- [9] 侯一民, 周慧琼, 王政一. 深度学习在语音识别中的研究进展综述. *计算机应用研究*, 2017, 34(8): 2241-2246
- [10] 陈游旻, 李飞, 舒继武. 大数据环境下的存储系统构建: 挑战、方法和趋势. *大数据*, 2019, 5(4): 27-40

- [11] Norman P. Jouppi, Cliff Young, Nishant Patil, David A. Patterson, Gaurav Agrawal, Raminder Bajwa, et al. In-Datcenter Performance Analysis of a Tensor Processing Unit. In: Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA 2017). Toronto, Canada, June 24-28, 2017: 1-12
- [12] Karen Simonyan, Andrew Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. In: Proceedings of 3rd International Conference on Learning Representations (ICLR 2015). San Diego, California, United States, May 7-9, 2015
- [13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. Deep Residual Learning for Image Recognition. In: Proceedings of 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2016). Las Vegas, Nevada, United States, June 27-30, 2016: 770-778
- [14] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, Zbigniew Wojna. Rethinking the Inception Architecture for Computer Vision. In: Proceedings of 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2016). Las Vegas, Nevada, United States, June 27-30, 2016: 2818-2826
- [15] Gao Huang, Zhuang Liu, Laurens van der Maaten, Kilian Q. Weinberger. Densely Connected Convolutional Networks. In: Proceedings of 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2017). Honolulu, HI, USA, July 21-26, 2017: 2261-2269
- [16] Dale, Robert. GPT-3: What's it good for? Natural Language Engineering. 2021, 27(1): 113-118
- [17] Matthew D. Zeiler, Rob Fergus. Visualizing and Understanding Convolutional Networks. In: Proceedings of 13th European Conference on Computer Vision (ECCV 2014). Zurich, Switzerland, September 6-12, 2014: 818-833
- [18] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, et al. Efficient Large-Scale Language Model Training on GPU Clusters using Megatron-LM. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC 2021). St. Louis Missouri, November 14 - 19, 2021: 1-15

- [19] Jinrong Guo, Wantao Liu, Wang Wang, Chunrong Yao, Jizhong Han, Ruixuan Li, et al. AccUDNN: A GPU Memory Efficient Accelerator for Training Ultra-deep Neural Networks. In: Proceedings of 2019 IEEE 37th International Conference on Computer Design (ICCD 2019). Abu Dhabi, United Arab Emirates, November 17-20, 2019: 65-72
- [20] Bojian Zheng, Nandita Vijaykumar, Gennady Pekhimenko. Echo: Compiler-based GPU Memory Footprint Reduction for LSTM RNN Training. In: Proceedings of 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA 2020). Valencia, Spain, May 30 - June 03, 2020: 1089-1102
- [21] 高赫然, 吴恒, 许源佳, 李修和, 王焘, 张文博. 面向深度学习训练的内存交换机制综述. 软件学报, 2022
- [22] 马玮良, 彭轩, 熊倩, 石宣化, 金海. 深度学习中的内存管理问题研究综述. 大数据, 2020, 6(4): 56-68
- [23] Minsoo Rhu; Natalia Gimelshein; Jason Clemons; Arslan Zulfiqar; Stephen W. Keckler. vDNN: Virtualized Deep Neural Networks for Scalable, Memory-Efficient Neural Network Design. In: Proceedings of 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2016). Taipei, Taiwan, October 15-19, 2016: 1-13
- [24] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen L. Song, et al. Superneurons: Dynamic GPU Memory Management for Training Deep Neural Networks. In: Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP 2018). Vienna, Austria, February 24-28, 2018: 41-53
- [25] Chen Meng, Minmin Sun, Jun Yang, Minghui Qiu, Yang Gu. Training Deeper Models by GPU Memory Optimization on TensorFlow. In: Proceedings of Advances in Neural Information Processing Systems 30(NIPS 2017). Long Beach, CA, USA, December 4-9, 2017: 7
- [26] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, et al. Tensorflow: A System for Large-scale Machine Learning. In: Proceedings of

- the 12th USENIX conference on Operating Systems Design and Implementation (OSDI 2016). Savannah GA USA, November 2-4, 2016, 16: 265–283
- [27] Xiaoming Chen, Danny Z. Chen, Yinhe Han, Xiaobo S. Hu. moDNN: Memory Optimal Deep Neural Network Training on Graphics Processing Units. *IEEE Transactions on Parallel and Distributed Systems*, 2019, 30(3): 646-661
- [28] Hai Jin, Bo Liu, Wenbin Jiang, Yang Ma, Xuanhua Shi, Bingsheng He, et al. Layer-Centric Memory Reuse and Data Migration for Extreme-Scale Deep Learning on Many-Core Architectures. *ACM Transactions on Architecture and Code Optimization*, 2018, 15(3): 1-26
- [29] Xuan Peng, Xuanhua Shi, Hulin Dai, Hai Jin, Weiliang Ma, Qian Xiong, et al. Capuchin: Tensor-based GPU Memory Management for Deep Learning. In: *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2020)*. Lausanne Switzerland, March 16-20, 2020: 891-905
- [30] Jonghyun Bae, Jongsung Lee, Yunho Jin, Sam Son, Shine Kim, Hakbeom Jang, et al. FlashNeuron: SSD-Enabled Large-Batch Training of Very Deep Neural Networks. In: *Proceedings of 19th USENIX Conference on File and Storage Technologies (FAST 2021)*. February 23-25, 2021: 387-401
- [31] Audrunas Gruslys, Remi Munos, Ivo Danihelka, Marc Lanctot, Alex Graves. Memory-Efficient Backpropagation Through Time. In: *Proceedings of Advances in Neural Information Processing Systems 29 (NIPS 2016)*. Barcelona, Spain, December 5-10, 2016: 4125-4133
- [32] Mitsuru Kusumoto, Takuya Inoue, Gentaro Watanabe, Takuya Akiba, Masanori Koyama. A Graph Theoretic Framework of Recomputation Algorithms for Memory-Efficient Backpropagation. In: *Proceedings of Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*. Vancouver, BC, Canada, December 8-14, 2019: 1161-1170
- [33] Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Joseph Gonzalez, et al. Checkmate: Breaking the Memory Wall with Optimal Tensor Rematerialization. In: *Proceedings of Machine Learning and Systems 2020 (MLSys 2020)*. Austin, TX, USA, March 2-4, 2020: 497-511

- [34] 雷杰, 高鑫, 宋杰, 王兴路, 宋明黎. 深度网络模型压缩综述. 软件学报, 2018, 29(2): 251-266
- [35] 高晗, 田育龙, 许封元, 仲盛. 深度学习模型压缩与加速综述. 软件学报, 2021, 32(1): 68-92.
- [36] Song Han, Jeff Pool, John Tran, William Dally. Learning both weights and connections for efficient neural network. In: Proceedings of Advances in Neural Information Processing Systems 28 (NIPS 2015). Montreal, Quebec, Canada, December 7-12, 2015: 1135-1143
- [37] Yiwen Guo, Anbang Yao, Yurong Chen. Dynamic Network Surgery for Efficient DNNs. In: Proceedings of Advances in Neural Information Processing Systems 29 (NIPS 2016). Barcelona, Spain, December 5-10, 2016: 1379-1387
- [38] Yunhe Wang, Chang Xu, Chunjing Xu, Chao Xu, Dacheng Tao. Learning Versatile Filters for Efficient Convolutional Neural Networks. In: Proceedings of Advances in Neural Information Processing Systems 29 (NeurIPS 2018). Montréal, Canada, December 3-8, 2018: 1615-1625
- [39] Ameya Prabhu, Girish Varma, Anoop Namboodiri. Deep Expander Networks: Efficient Deep Networks from Graph Theory. In: Proceedings of the European Conference on Computer Vision (ECCV 2018). Munich, Germany, September 8-14, 2018: 20-35
- [40] Animesh Jain, Amar Phanishayee, Jason Mars, Lingjia Tang, Gennady Pekhimenko. Gist: Efficient Data Encoding for Deep Neural Network Training. In: Proceedings of 45th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA 2018). Los Angeles, CA, USA, June 1-6, 2018: 776-789
- [41] Song Han, Huizi Mao, William J. Dally. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In: Proceedings of 4th International Conference on Learning Representations (ICLR 2016). San Juan, Puerto Rico, May 2-4, 2016: 00149
- [42] Minsoo Rhu, Mike O'Connor, Niladrish Chatterjee, Jeff Pool, Youngeun Kwon, Stephen W. Keckler. Compressing DMA Engine: Leveraging Activation Sparsity for Training Deep Neural Networks. In: Proceedings of 2018 IEEE International

- Symposium on High Performance Computer Architecture (HPCA 2018). Vienna, Austria, February 24-28, 2018: 78-91
- [43] Yann Lecun, Leon Bottou, Yoshua Bengio, Patrick Haffner. Gradient-based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 1998, 86(11): 2278-2324
- [44] Xiangui Yu, Nan K. Loh, William C. Miller. A New Acceleration Technique for the Backpropagation Algorithm. In: *Proceedings of IEEE International Conference on Neural Networks(ICNN 1993)*. San Francisco, CA, USA, March 28-April 1,1993: 1157-1161
- [45] Kihyuk Sohn, Honglak Lee, Xincheng Yan. Learning Structured Output Representation using Deep Conditional Generative Models. In: *Proceedings of Advances in Neural Information Processing Systems 28 (NIPS 2015)*. Montreal, Quebec, Canada, December 7-12, 2015: 3483-3491
- [46] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, et al. Large Scale Distributed Deep Networks. In: *Proceedings of Advances in Neural Information Processing Systems 25 (NIPS 2012)*. Lake Tahoe, Nevada, United States, December 3-6, 2012: 1223-1231
- [47] Linghao Song, Fan Chen, Youwei Zhuo, Xuehai Qian, Hai Li, Yiran Chen. Accpar: Tensor Partitioning for Heterogeneous Deep Learning Accelerator Arrays. In: *Proceedings of 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA 2020)*. San Diego, CA, USA, February 22-26, 2020: 342-355
- [48] Linghao Song, Jiachen Mao, Youwei Zhuo, Xuehai Qian, Hai Li, Yiran Chen. Hypar: Towards Hybrid Parallelism for Deep Learning Accelerator Array. In: *Proceedings of 2019 IEEE International Symposium on High Performance Computer Architecture (HPCA 2019)*. Washington, DC, USA, February 16-20, 2019: 56-68
- [49] 亢良伊, 王建飞, 刘杰, 叶丹. 可扩展机器学习的并行与分布式优化算法综述. *软件学报*, 2018, 29(1): 109-130

- [50] Arkaprava Basu, Jayneel Gandhi, Jichuan Chang, Mark D. Hill, Michael M. Swift. Efficient Virtual Memory for Big Memory Servers. ACM SIGARCH Computer Architecture News, 2013, 41(3): 237-248
- [51] M. Kandemir, Feihul Li, Guilin Chen, Guangyu Chen, O. Ozturk. Studying Storage-Recomputation Tradeoffs in Memory-Constrained Embedded Processing. In: Proceedings of Design, Automation and Test in Europe (DATE 2005). Munich, Germany, March 07-11, 2005: 1026-1031
- [52] Samuel R. Bulò, Lorenzo Porzi, Peter Kotschieder. In-Place Activated BatchNorm for Memory-Optimized Training of DNNs. In: Proceedings of 2018 IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2018). Salt Lake City, UT, USA, June 18-22, 2018: 5639-5647
- [53] Mark S. Johnstone, Paul R. Wilson. The Memory Fragmentation Problem: Solved? ACM SIGPLAN Notices, 1999, 34(3): 26-36
- [54] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: Proceedings of Advances in Neural Information Processing Systems 32 (NeurIPS 2019). Vancouver, BC, Canada, December 8-14, 2019: 8024-8035.
- [55] Isaac Gelado, Michael Garland. Throughput-Oriented GPU Memory Allocation. In: Proceedings of the 24th Symposium on Principles and Practice of Parallel Programming (PPoPP 2019). Washington District of Columbia, February 16-20, 2019: 27-37
- [56] Donglin Yang, Dazhao Cheng. Efficient GPU Memory Management for Nonlinear DNNs. In: Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing (HPDC 2020). Stockholm, Sweden, June 23-26, 2020: 185-196
- [57] Shriram S.B., Anshuj Garg, Purushottam Kulkarni. Dynamic Memory Management for GPU-Based Training of Deep Neural Networks. In: Proceedings of 2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS 2019). Rio de Janeiro, Brazil, May 20-24, 2019: 200-209
- [58] David A. Beckingsale, Marty J. McFadden, Johann P. S. Dahm, Ramesh Pankajakshan, Richard D. Hornung. Umpire: Application-focused management and

- coordination of complex hierarchical memory. IBM Journal of Research and Development, 2019, 64(3/4): 00:1-00:10
- [59] Shaohuai Shi, Qiang Wang, Pengfei Xu, Xiaowen Chu. Benchmarking State-of-the-Art Deep Learning Software Tools. In: Proceedings of 7th International Conference on Cloud Computing and Big Data (CCBD 2016). Macau, China, November 16-18, 2016: 99-104
- [60] Gregory Cohen, Saeed Afshar, Jonathan Tapson; André van Schaik. EMNIST: Extending MNIST to Handwritten Letters. In: Proceedings of 2017 International Joint Conference on Neural Networks (IJCNN 2017). Anchorage, AK, USA, May 14-19, IEEE, 2017: 2921-2926
- [61] Henggang Cui, Alexey Tumanov, Jinliang Wei, Lianghong Xu, Wei Dai, Jesse Haber-Kucharsky, et al. Exploiting Iterative-ness for Parallel ML Computations. In: Proceedings of ACM Symposium on Cloud Computing (SOCC 2014). Seattle, WA, USA, November 3 - 5, ACM, 2014: 1-14
- [62] Jinliang Wei, Wei Dai, Aurick Qiao, Qirong Ho, Henggang Cui, Gregory R. Ganger, et al. Managed Communication and Consistency for Fast Data Parallel Iterative Analytics. In: Proceedings of the Sixth ACM Symposium on Cloud Computing (SOCC 2015). Kohala Coast, Hawaii, August 27 - 29, ACM, 2015: 381-394
- [63] Jiuxing Liu, Jiesheng Wu & Dhabaleswar K. Panda. High Performance RDMA-Based MPI Implementation over InfiniBand. International Journal of Parallel Programming, 2004, 32(3): 167-198
- [64] Anuj Kalia, Michael Kaminsky, David G. Andersen. Using RDMA Efficiently for Key-Value Services. ACM SIGCOMM Computer Communication Review, 2015, 44(4): 295-306.